



UFABC

Universidade Federal do ABC
Centro de Matemática, Computação e Cognição

Estrutura de Dados

Monael Pinheiro Ribeiro, D.Sc.



UFABC

Universidade Federal do ABC
Centro de Matemática, Computação e Cognição

Árvores

Monael Pinheiro Ribeiro, D.Sc.

Árvores

Definição

árvore s. f. 1. Bot. Vegetal lenhoso, de tronco alto, que se ramifica a maior ou menor altura do solo. 2. Mec., Autom. Peça, geralmente alongada, que serve para transmitir ou receber o movimento nas máquinas. 3. Náut. Colocar mastros: A. uma nau.

árvore s. f. 1. Vegetal de tronco lenhoso cujos ramos só saem a certa altura do solo. 2. Qualquer representação ou esquematização que, pelas suas ramificações, lembre essa planta. 3. Peça principal que transmite o movimento de uma máquina. 4. Mastro completo (de navio). 5. *Poét.* Navio.

Árvores

Definição



árvore s. f. 1. Bot. Vegetal lenhoso, de tronco alto, que se ramifica a maior ou menor altura do solo. 2. Mec., Autom. Peça, geralmente alongada, que serve para transmitir ou receber o movimento nas máquinas. 3. Náut. Colocar mastros: A. uma nau.

árvore s. f. 1. Vegetal de tronco lenhoso cujos ramos só saem a certa altura do solo. 2. Qualquer representação ou esquematização que, pelas suas ramificações, lembre essa planta. 3. Peça principal que transmite o movimento de uma máquina. 4. Mastro completo (de navio). 5. *Poét.* Navio.

Árvores

Definição



árvore: no contexto da programação e ciência da computação, é uma estrutura de dados que herda as características das topologias em árvore.

Conceitualmente diferente das listas encadeadas, em que os dados se encontram numa sequência, nas árvores os dados estão dispostos de forma hierárquica.

A árvore é composta por um elemento principal chamado raiz, que possui ligações para outros elementos, que são denominados de galhos ou filhos. Estes galhos levam a outros elementos que também possuem outros galhos. O elemento que não possui galhos é conhecido como folha ou nó terminal.

Árvores

Exemplos:



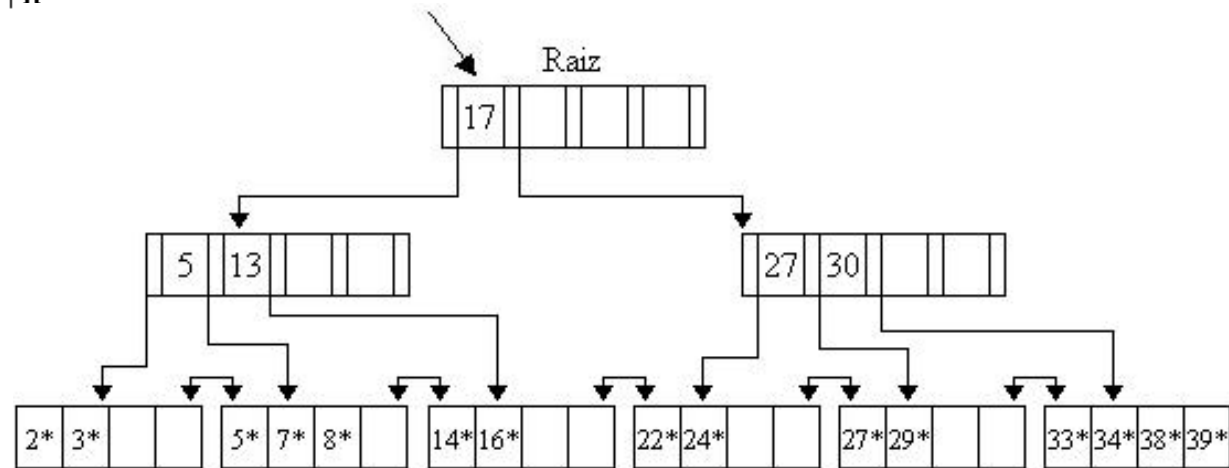
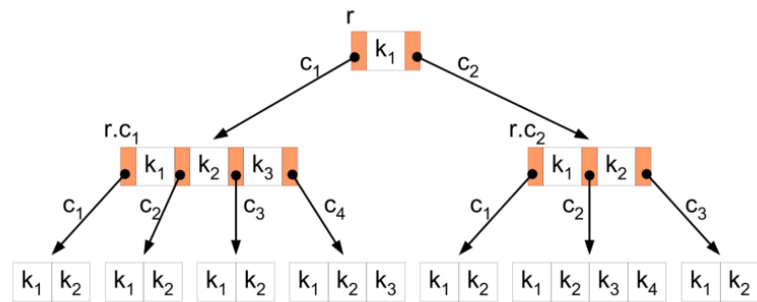
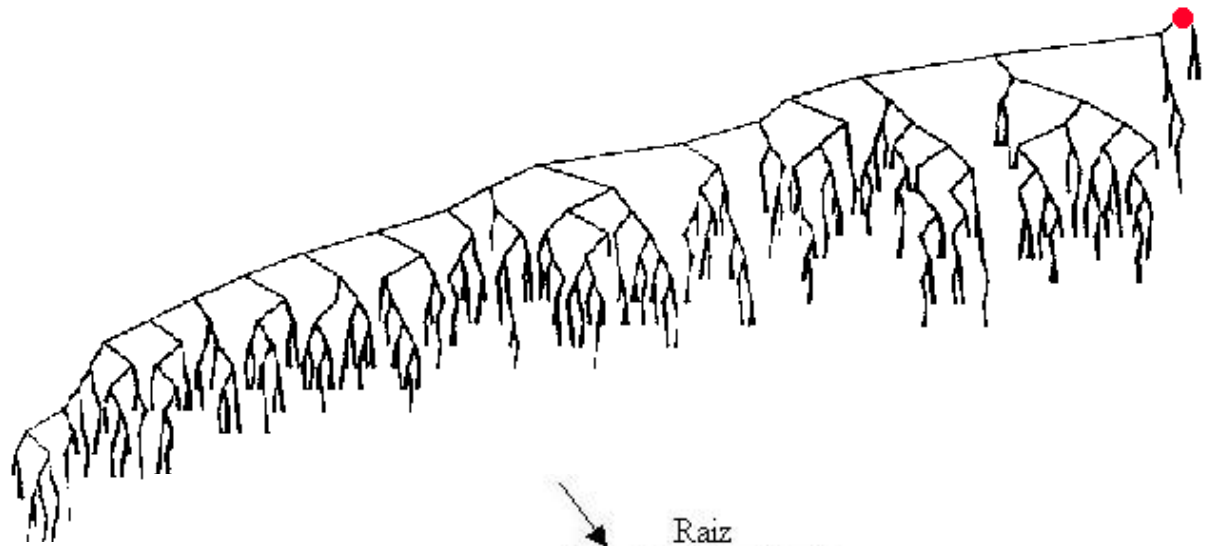
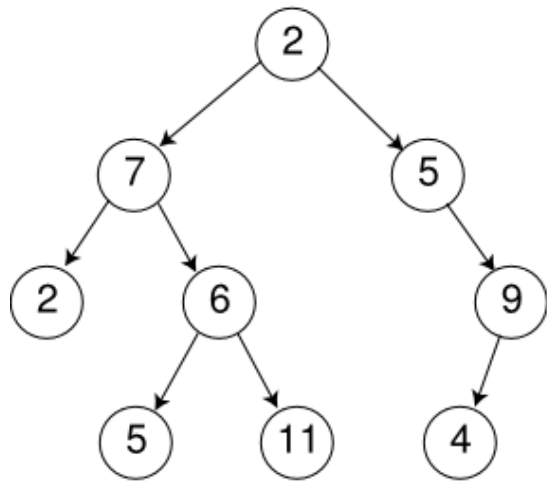
Árvores

Exemp



Árvores

Exemplos:



Árvores

Uma árvore enraizada T , ou simplesmente árvore, é um conjunto finito de elementos denominados nós tais que:

$T = \emptyset$ é a árvore dita vazia. Ou

Existe um nó especial r , chamado raiz de T ;

os nós restantes constituem um único conjunto, que pode ser vazio ou não, que são divididos em b ($b \geq 1$) conjuntos distintos não vazios que são as subárvores de r , cada subárvore a qual é, por sua vez, uma árvore.

Árvores

Uma árvore enraizada T , ou simplesmente árvore, é um conjunto finito de elementos denominados nós tais que:

$T = \emptyset$ é a árvore dita vazia. Ou

Existe um nó especial r , chamado raiz de T ;

os nós restantes constituem um único conjunto, que pode ser vazio ou não, que são divididos em b ($b \geq 1$) conjuntos distintos não vazios que são as subárvores de r , cada subárvore a qual é, por sua vez, uma árvore.



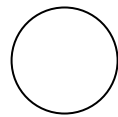
Árvores

Uma árvore enraizada T , ou simplesmente árvore, é um conjunto finito de elementos denominados nós tais que:

$T = \emptyset$ é a árvore dita vazia. Ou

Existe um nó especial r , chamado raiz de T ;

os nós restantes constituem um único conjunto, que pode ser vazio ou não, que são divididos em b ($b \geq 1$) conjuntos distintos não vazios que são as subárvores de r , cada subárvore a qual é, por sua vez, uma árvore.



Árvores

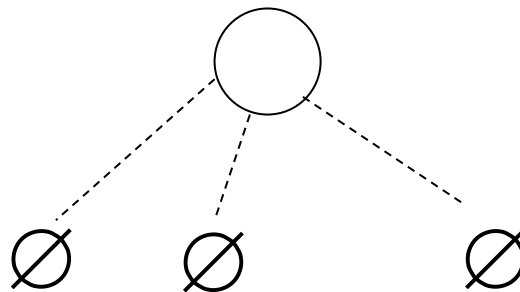
Uma árvore enraizada T , ou simplesmente árvore, é um conjunto finito de elementos denominados nós tais que:

$T = \emptyset$ é a árvore dita vazia. Ou

Existe um nó especial r , chamado raiz de T ;

os nós restantes constituem um único conjunto, que pode ser vazio ou não.

Esses são divididos em b ($b \geq 1$) conjuntos distintos não vazios que são as subárvores de r , cada subárvore a qual é, por sua vez, uma árvore.



Árvores

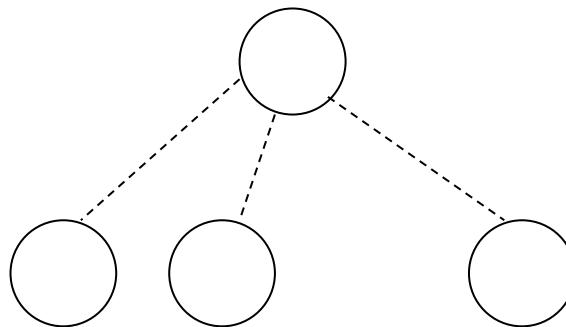
Uma árvore enraizada T , ou simplesmente árvore, é um conjunto finito de elementos denominados nós tais que:

$T = \emptyset$ é a árvore dita vazia. Ou

Existe um nó especial r , chamado raiz de T ;

os nós restantes constituem um único conjunto, que pode ser vazio ou não.

Esses são divididos em b ($b \geq 1$) conjuntos distintos não vazios que são as subárvores de r , cada subárvore a qual é, por sua vez, uma árvore.



Árvores

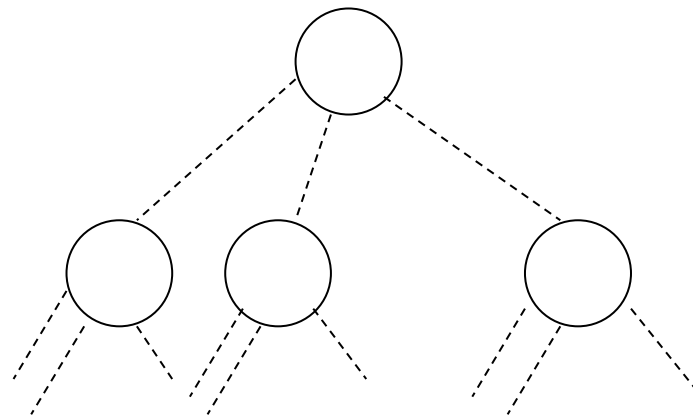
Uma árvore enraizada T , ou simplesmente árvore, é um conjunto finito de elementos denominados nós tais que:

$T = \emptyset$ é a árvore dita vazia. Ou

Existe um nó especial r , chamado raiz de T ;

os nós restantes constituem um único conjunto, que pode ser vazio ou não.

Esses são divididos em b ($b \geq 1$) conjuntos distintos não vazios que são as subárvores de r , cada subárvore a qual é, por sua vez, uma árvore.



b conjuntos não vazios!
Que por sua vez, são árvores

Árvores

Motivação:

Por que usar árvores?

Árvores

Motivação:

Por que usar árvores?

Custo

Árvores

Motivação:

Por que usar árvores?

Custo

É uma ED muito eficiente para armazenar informações

Árvores

Motivação:

Por que usar árvores?

Custo

É uma ED muito eficiente para armazenar informações

É particularmente adequada quando existe a necessidade de considerar todos ou alguma combinação de requisitos, tais como:

Árvores

Motivação:

Por que usar árvores?

Custo

É uma ED muito eficiente para armazenar informações

É particularmente adequada quando existe a necessidade de considerar todos ou alguma combinação de requisitos, tais como:

- Acesso Direto e Seqüencial eficientes

- Facilidade de Inserção e retirada de itens

- Otimização da utilização da memória

Árvores

Motivação:

Por que usar árvores?

Custo

Operação	Listas	Árvores
Inserção	$O(n)$	$O(\log_b n)$
Remoção	$O(n)$	$O(\log_b n)$
Busca	$O(n)$	$O(\log_b n)$

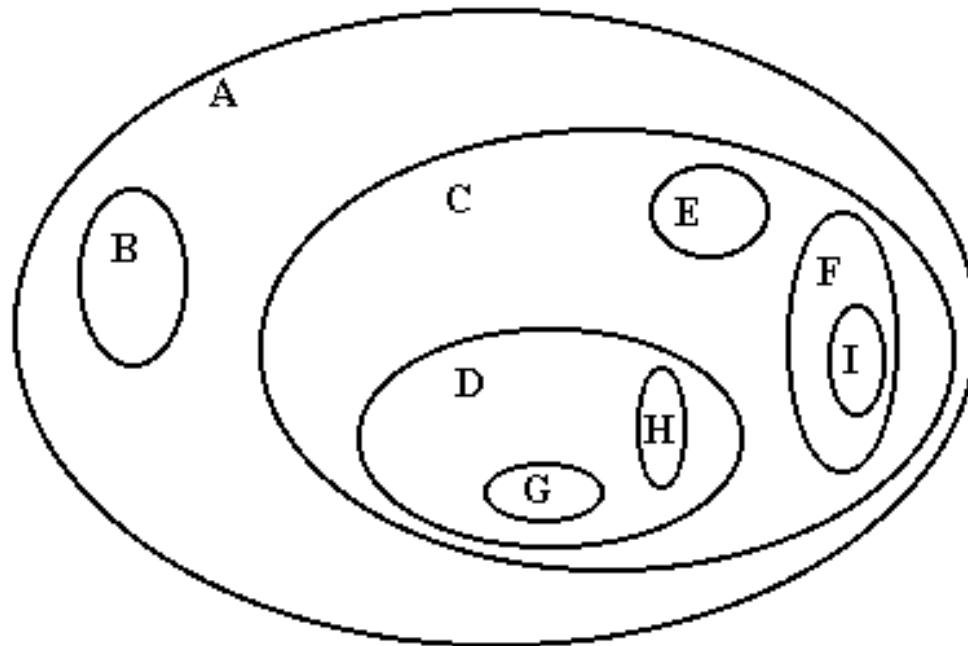
Árvores

Exemplos de representações:

Parênteses Aninhados

(A (B) (C (D (G) (H)) (E) (F (I)))))

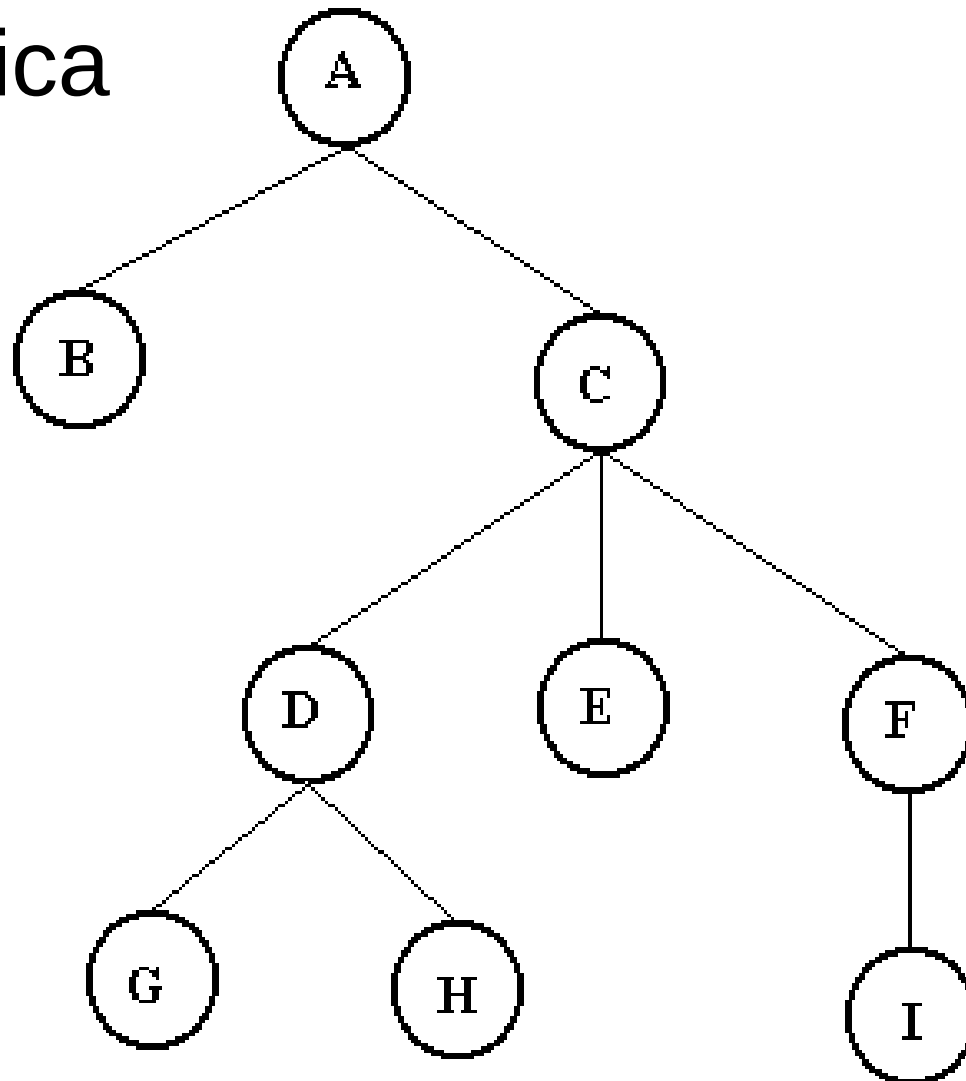
Diagrama de Inclusão



Árvores

Exemplos de representações:

Hierárquica



Árvores

Definições

Nós filhos, pais, tios, irmãos e avôs

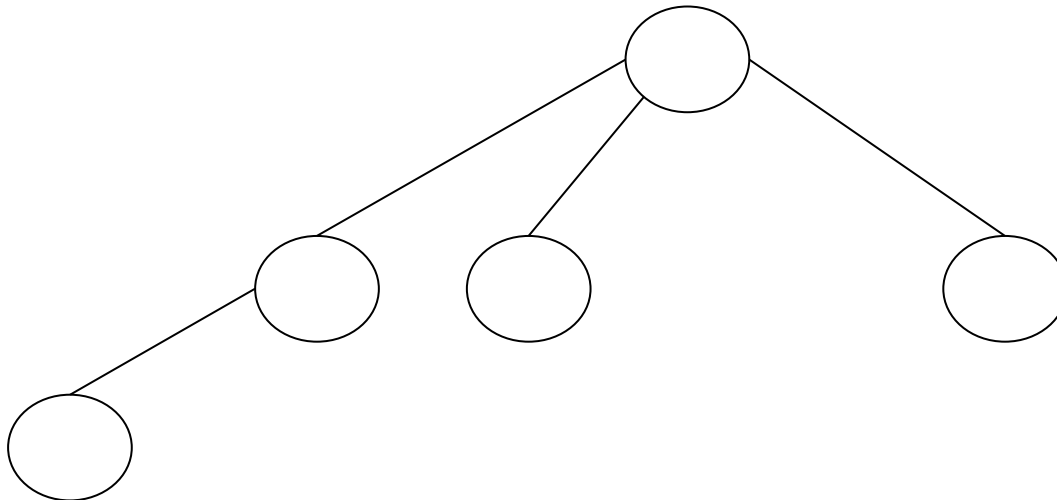
Seja v o nó raiz da subárvore T .

Os nós raízes $w_1, w_2, \dots, w_j$ das subárvores de v são chamados filhos de v .

v é chamado pai de $w_1, w_2, \dots, w_j$.

Os nós $w_1, w_2, \dots, w_j$ são irmãos.

Se z é filho de w_1 então w_2 é tio de z e v é avô de z .



Árvores

Definições

Nós filhos, pais, tios, irmãos e avôs

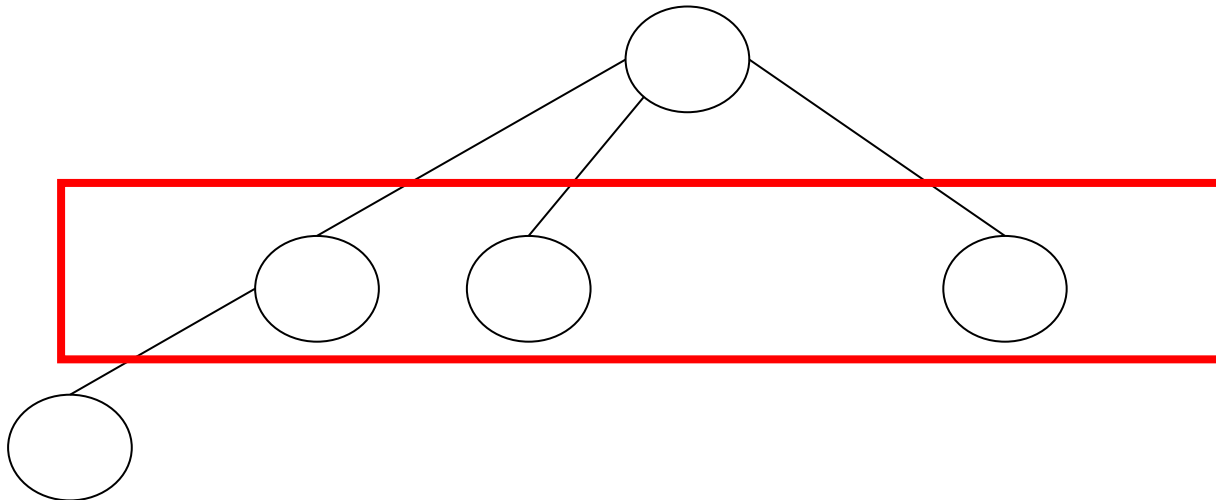
Seja v o nó raiz da subárvore T .

Os nós raízes $w_1, w_2, \dots, w_j$ das subárvores de v são chamados filhos de v .

v é chamado pai de $w_1, w_2, \dots, w_j$.

Os nós $w_1, w_2, \dots, w_j$ são irmãos.

Se z é filho de w_1 então w_2 é tio de z e v é avô de z .



Filhos de v

Árvores

Definições

Nós filhos, pais, tios, irmãos e avôs

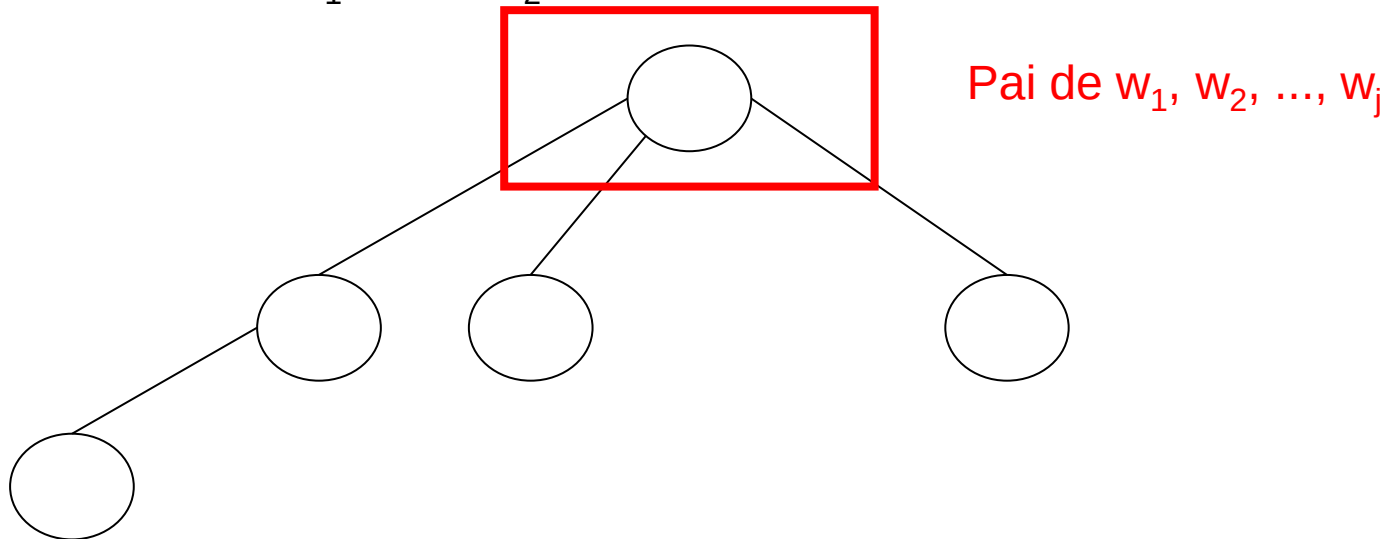
Seja v o nó raiz da subárvore T .

Os nós raízes $w_1, w_2, \dots, w_j$ das subárvores de v são chamados filhos de v .

v é chamado pai de $w_1, w_2, \dots, w_j$.

Os nós $w_1, w_2, \dots, w_j$ são irmãos.

Se z é filho de w_1 então w_2 é tio de z e v é avô de z .



Árvores

Definições

Nós filhos, pais, tios, irmãos e avôs

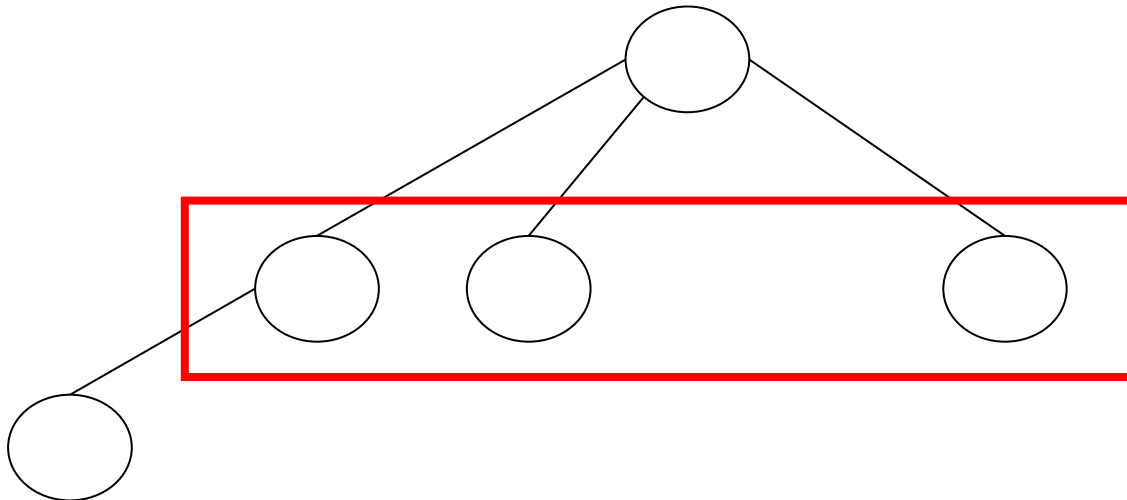
Seja v o nó raiz da subárvore T .

Os nós raízes $w_1, w_2, \dots, w_j$ das subárvores de v são chamados filhos de v .

v é chamado pai de $w_1, w_2, \dots, w_j$.

Os nós $w_1, w_2, \dots, w_j$ são irmãos.

Se z é filho de w_1 então w_2 é tio de z e v é avô de z .



Irmãos entre si

Árvores

Definições

Nós filhos, pais, tios, irmãos e avôs

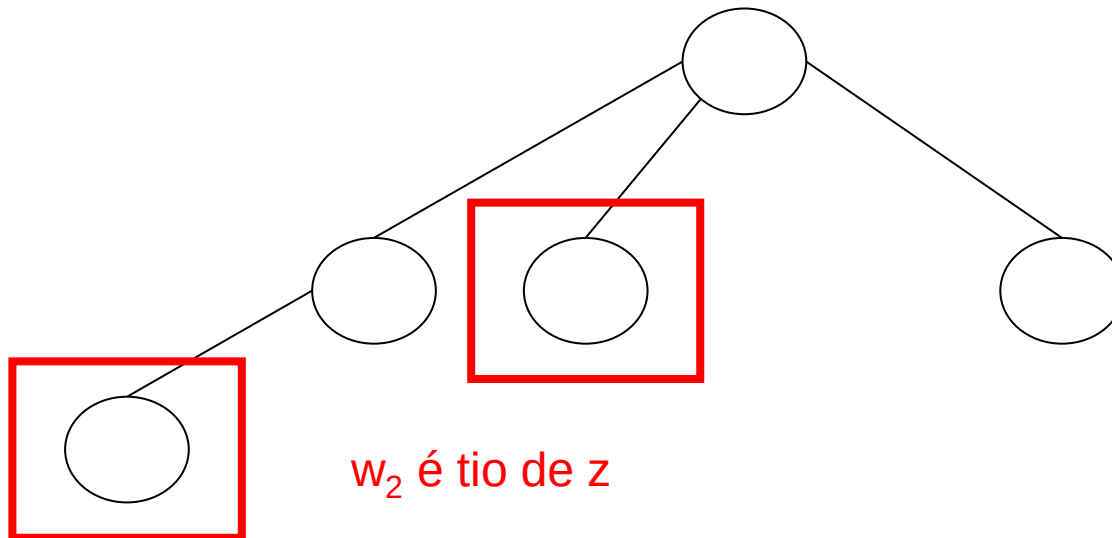
Seja v o nó raiz da subárvore T .

Os nós raízes $w_1, w_2, \dots, w_j$ das subárvores de v são chamados filhos de v .

v é chamado pai de $w_1, w_2, \dots, w_j$.

Os nós $w_1, w_2, \dots, w_j$ são irmãos.

Se z é filho de w_1 então w_2 é tio de z e v é avô de z .



Árvores

Definições

Nós filhos, pais, tios, irmãos e avôs

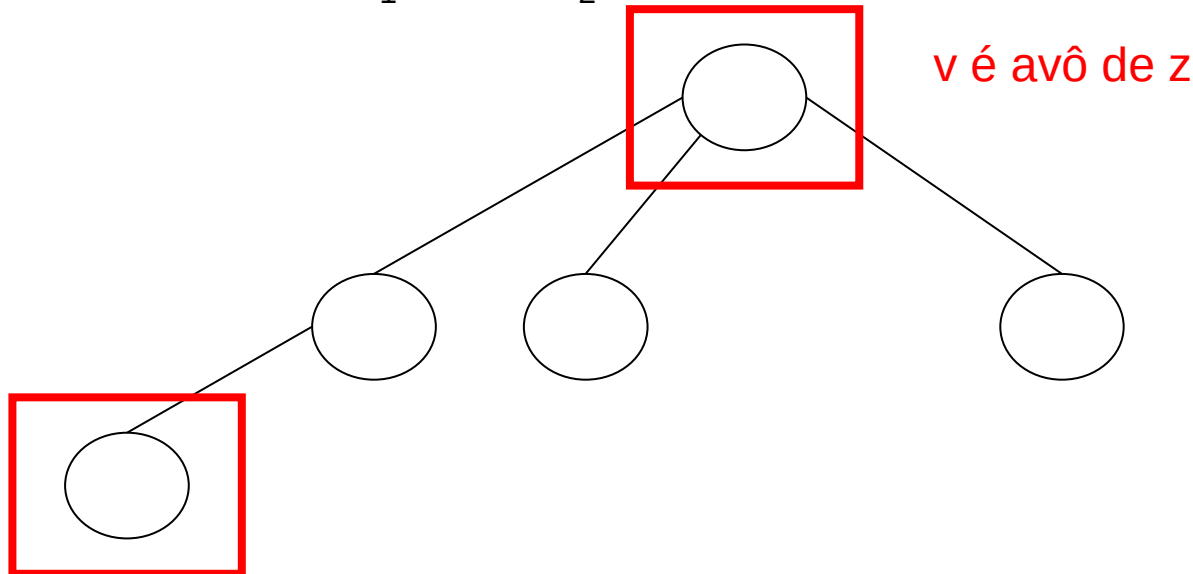
Seja v o nó raiz da subárvore T .

Os nós raízes $w_1, w_2, \dots, w_j$ das subárvores de v são chamados filhos de v .

v é chamado pai de $w_1, w_2, \dots, w_j$.

Os nós $w_1, w_2, \dots, w_j$ são irmãos.

Se z é filho de w_1 então w_2 é tio de z e v é avô de z .



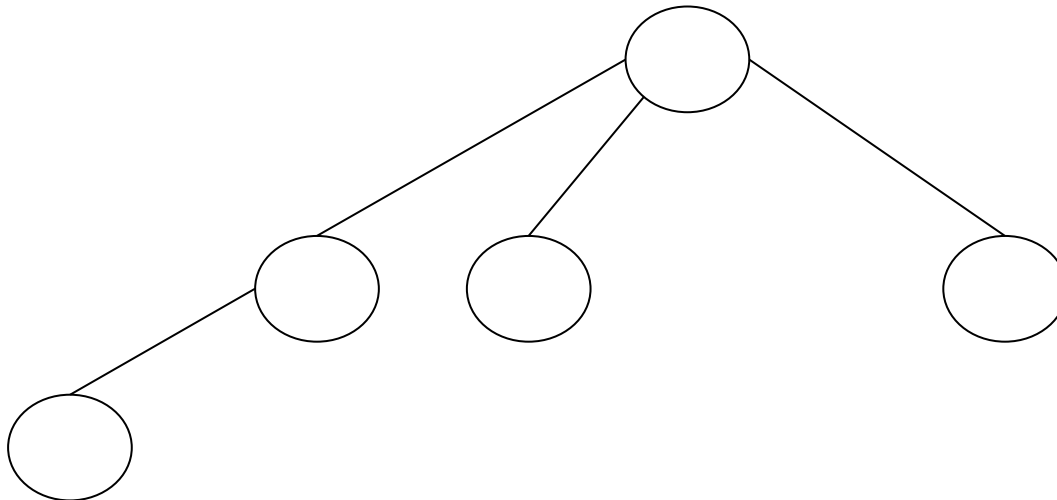
Árvores

Definições

Grau de saída, descendente e ancestral

O número de filhos de um nó é chamado grau de saída desse nó.

Se x pertence à subárvore de v , então, x é descendente de v e v é ancestral de x .



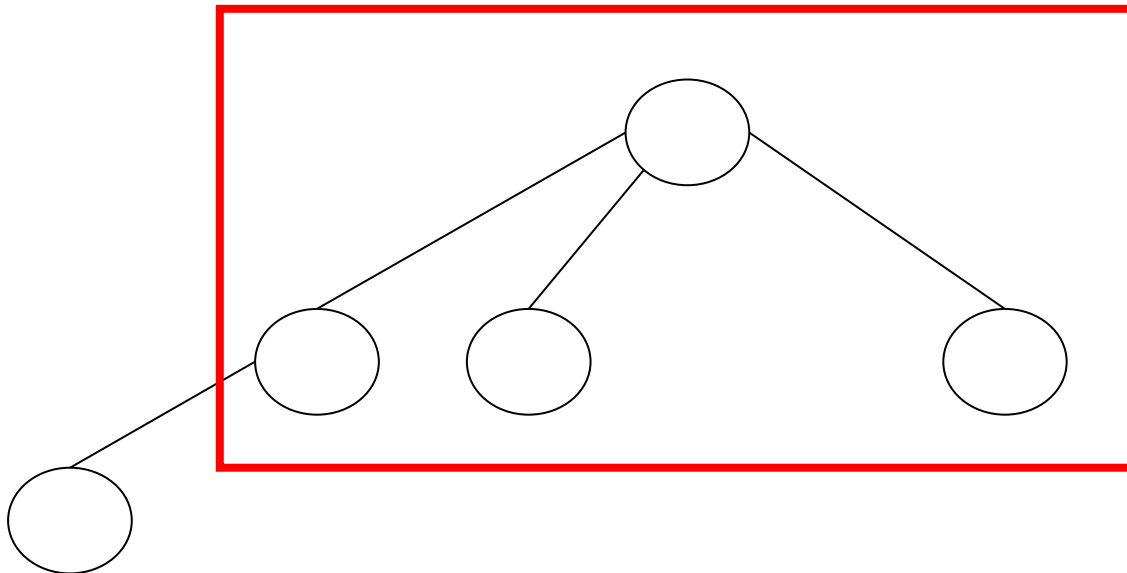
Árvores

Definições

Grau de saída, descendente e ancestral

O número de filhos de um nó é chamado grau de saída desse nó.

Se x pertence à subárvore de v , então, x é descendente de v e v é ancestral de x .



O nó v tem grau de saída j

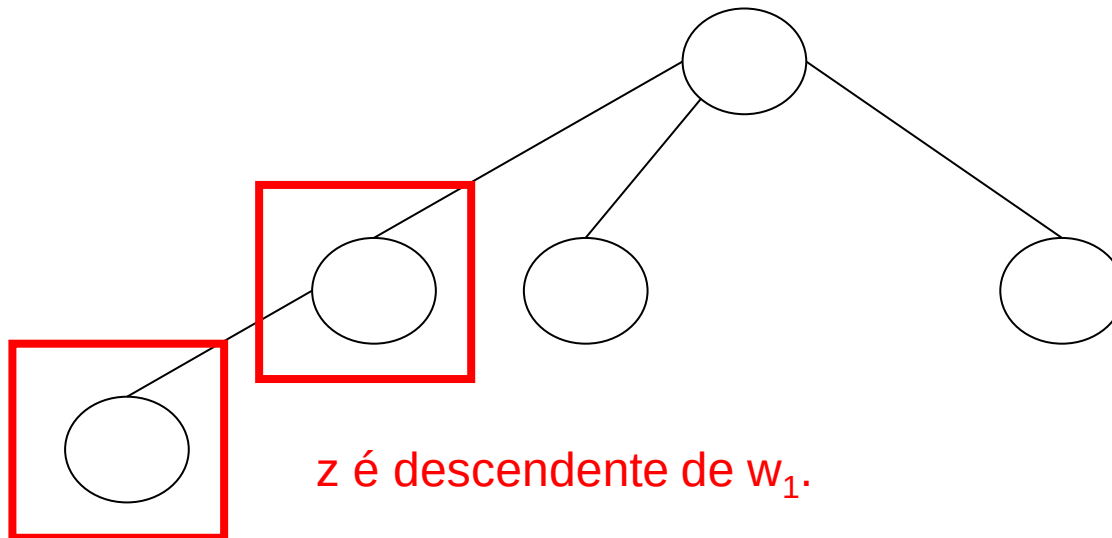
Árvores

Definições

Grau de saída, descendente e ancestral

O número de filhos de um nó é chamado grau de saída desse nó.

Se x pertence à subárvore de v , então, x é descendente de v e v é ancestral de x .



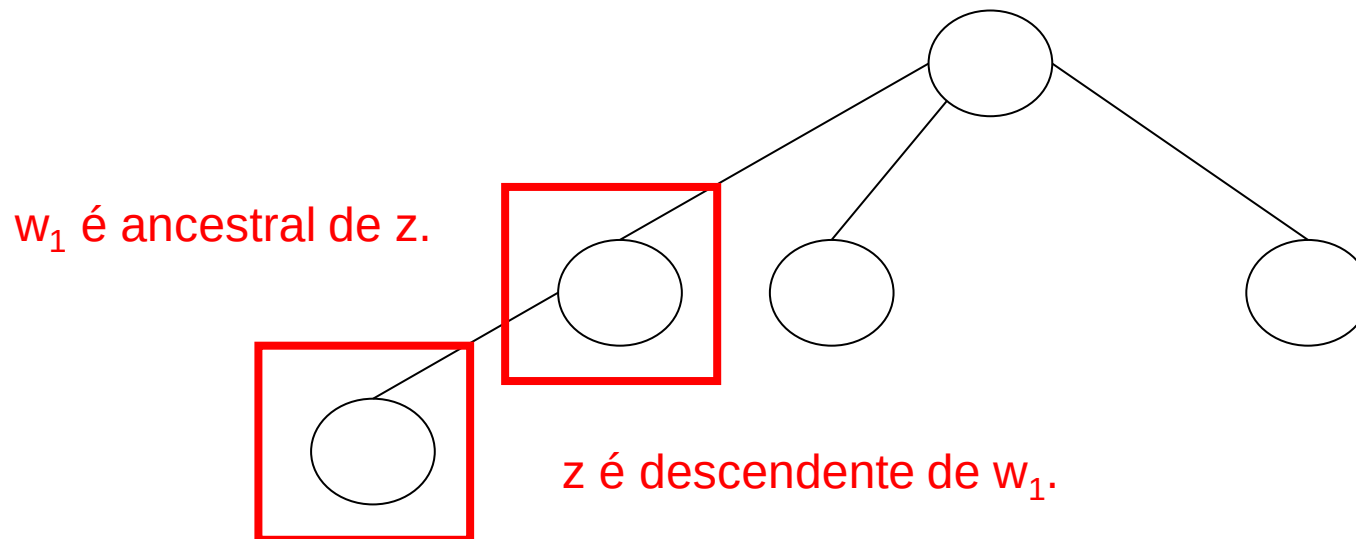
Árvores

Definições

Grau de saída, descendente e ancestral

O número de filhos de um nó é chamado grau de saída desse nó.

Se x pertence à subárvore de v , então, x é descendente de v e v é ancestral de x .



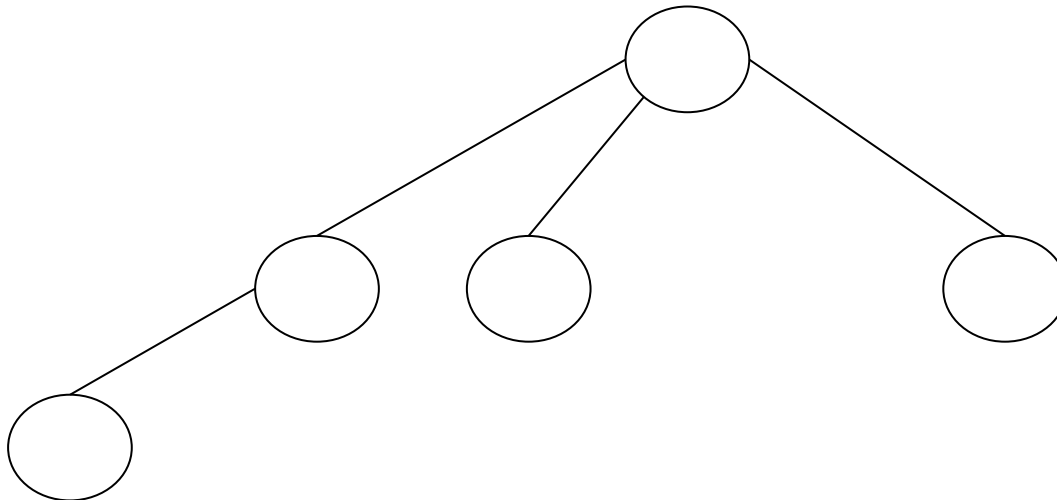
Árvores

Definições

Nó folha e Nó interior

Um nó que não possui descendentes é chamado de nó folha, ou seja, um nó folha é aquele com grau de saída nulo.

Um nó que não é folha (isto é, possui grau de saída diferente de zero) é chamado nó interior ou nó interno.



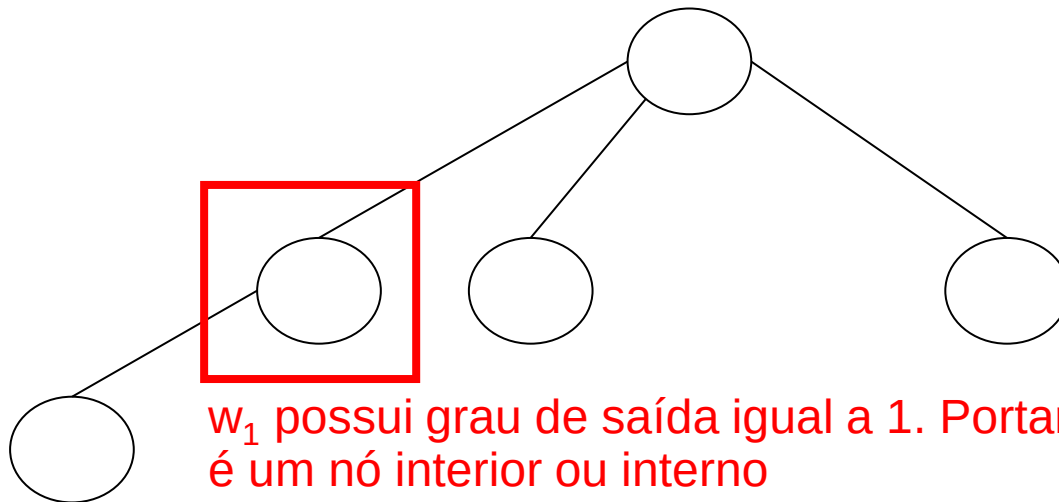
Árvores

Definições

Nó folha e Nó interior

Um nó que não possui descendentes é chamado de nó folha, ou seja, um nó folha é aquele com grau de saída nulo.

Um nó que não é folha (isto é, possui grau de saída diferente de zero) é chamado nó interior ou nó interno.



w_1 possui grau de saída igual a 1. Portanto, é um nó interior ou interno

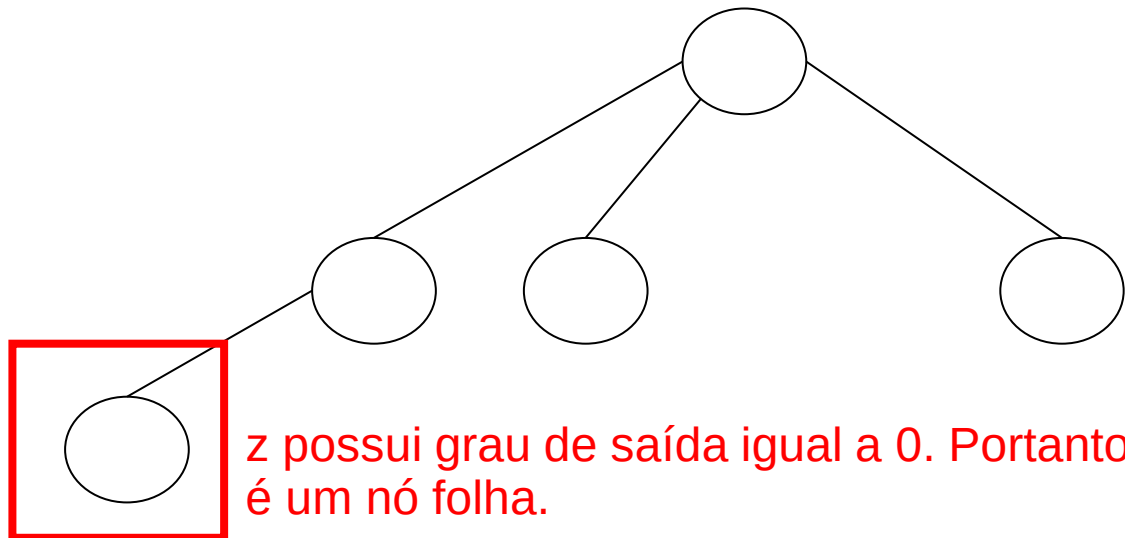
Árvores

Definições

Nó folha e Nó interior

Um nó que não possui descendentes é chamado de nó folha, ou seja, um nó folha é aquele com grau de saída nulo.

Um nó que não é folha (isto é, possui grau de saída diferente de zero) é chamado nó interior ou nó interno.

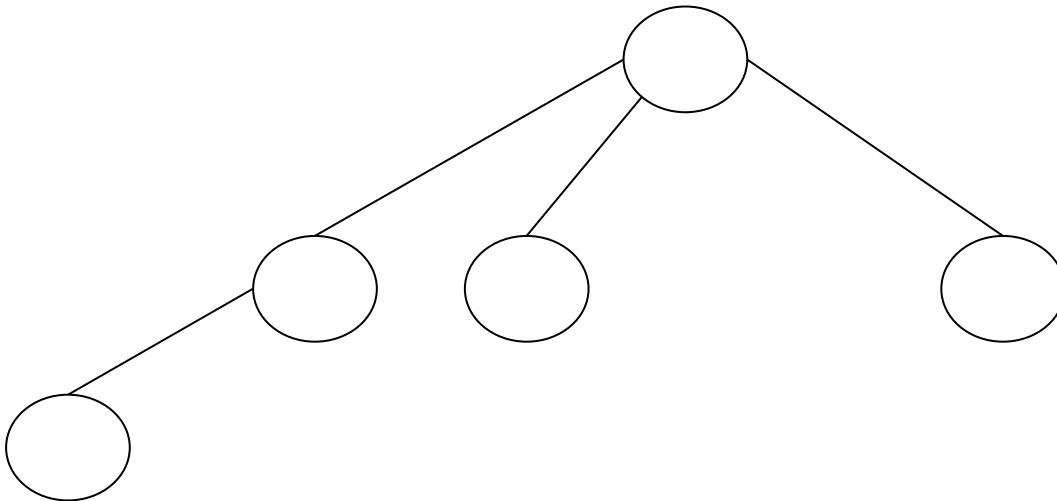


Árvores

Definições

Grau de uma árvore

O grau de uma árvore é o máximo entre os graus de seus nós.

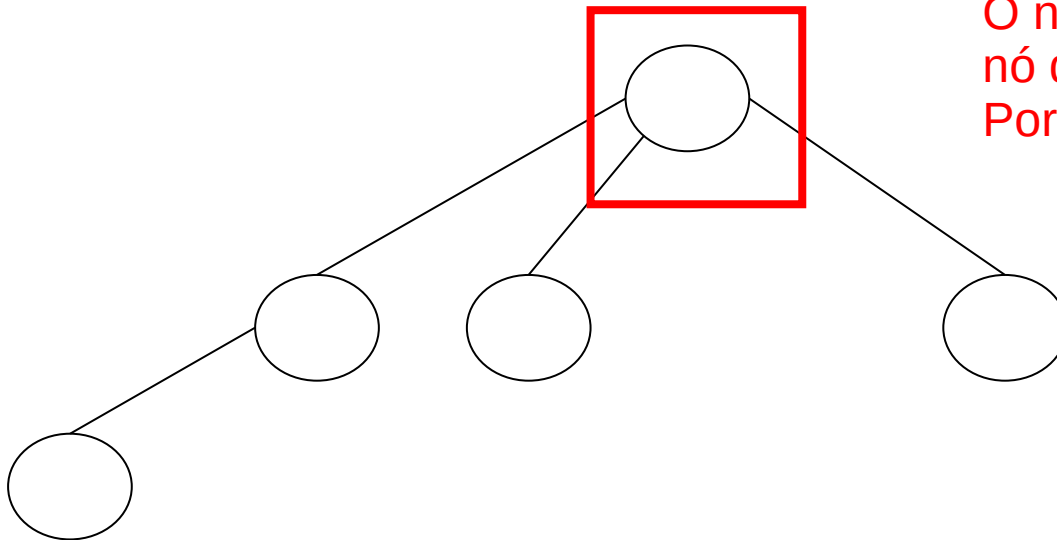


Árvores

Definições

Grau de uma árvore

O grau de uma árvore é o máximo entre os graus de seus nós.



O nó v de grau de saída j , $j > 2$, é o nó de maior grau na árvore. Portanto, j é o grau da árvore.

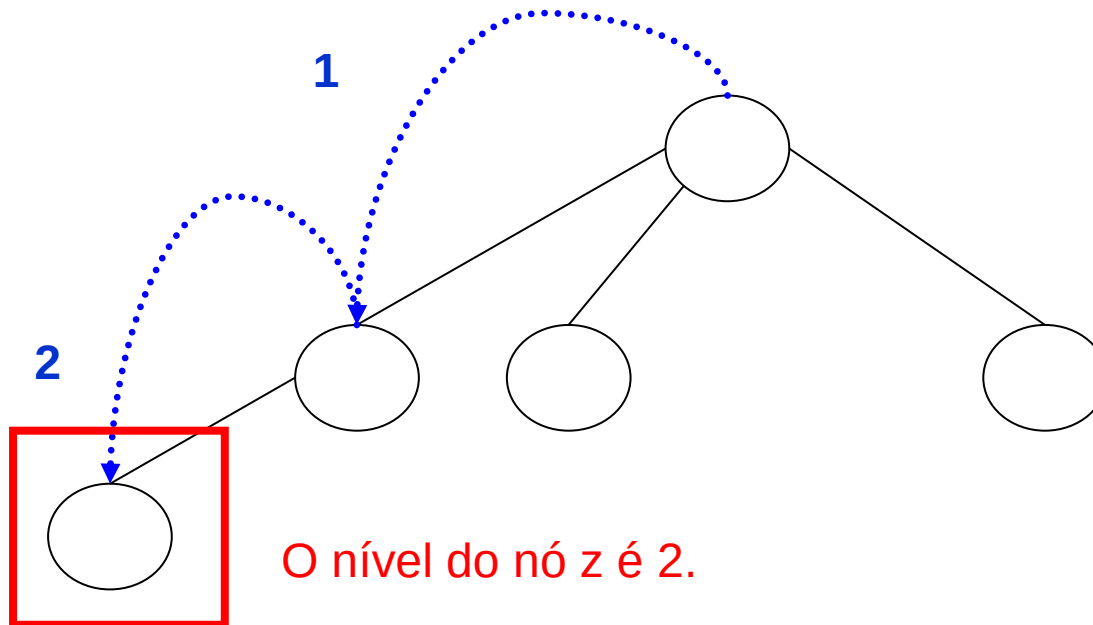
Árvores

Definições

Nível de um Nó e Altura de um Nó

O nível ou profundidade, de um nó é o número de nós do caminho da raiz até o nó. O nível da raiz, é portanto, 0.

A altura de um nó v é o número de nós no maior caminho de v até um de seus descendentes. As folhas têm altura 0.



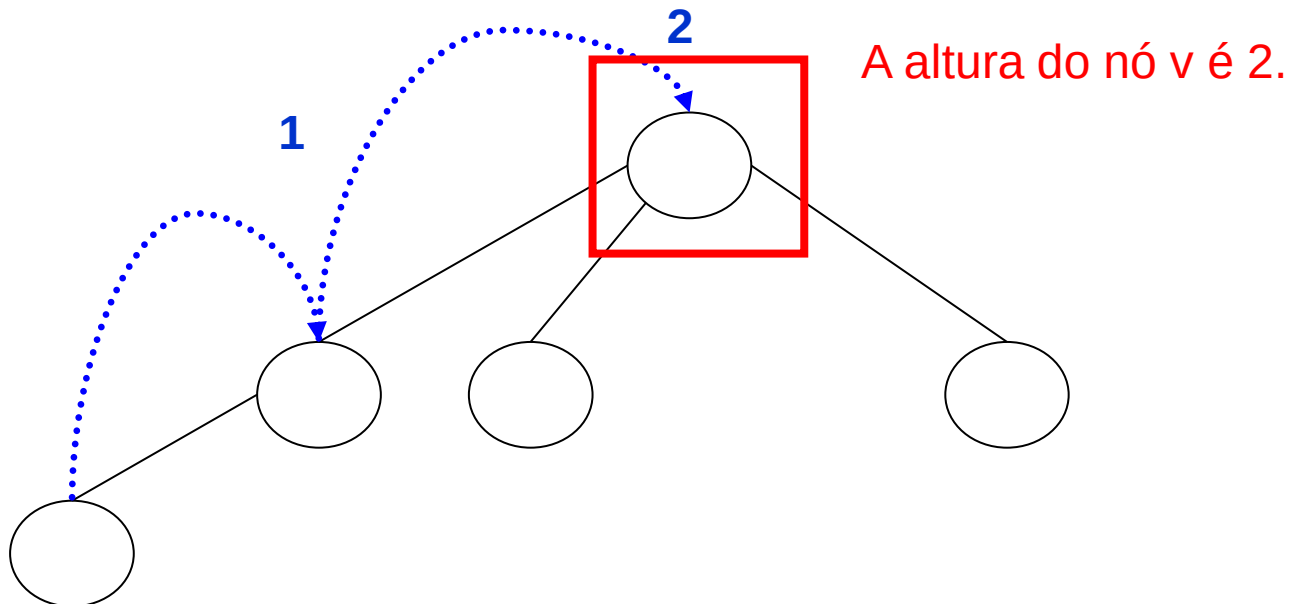
Árvores

Definições

Nível de um Nó e Altura de um Nó

O nível ou profundidade, de um nó é o número de nós do caminho da raiz até o nó. O nível da raiz, é portanto, 0.

A altura de um nó v é o número de nós no maior caminho de v até um de seus descendentes. As folhas têm altura 0.



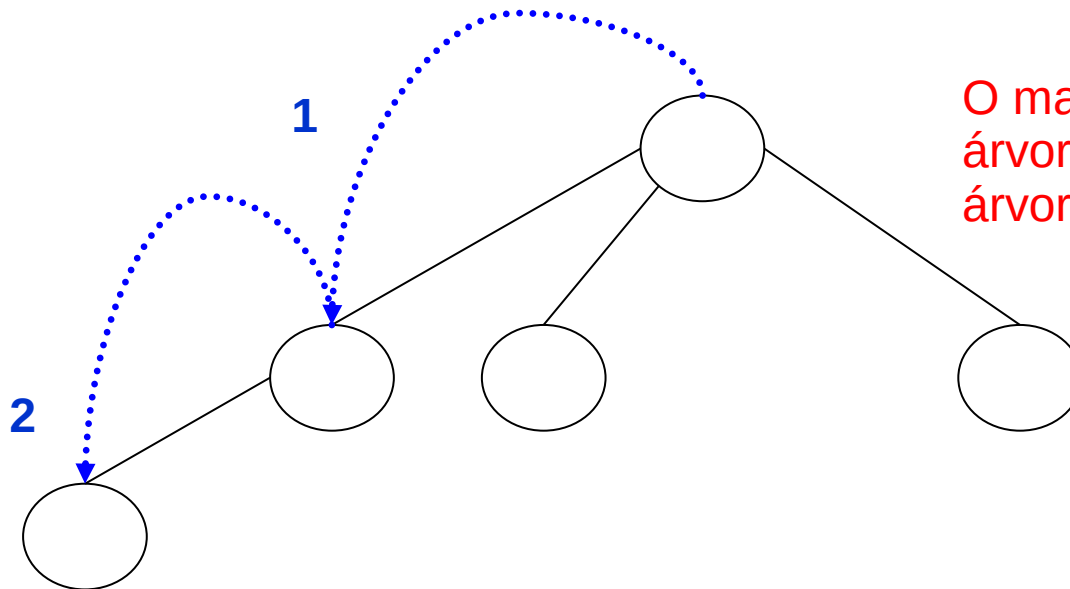
Árvores

Definições

Nível da Raiz e Altura de uma Árvore

O nível da raiz é 0.

A altura de uma árvore é igual ao máximo nível de seus nós. Representa-se a altura de árvore T por $h(T)$.



O maior nível entre os nós da árvore é 2, portanto a altura da árvore é 2.

Árvores

Definições

Árvore Ordenada

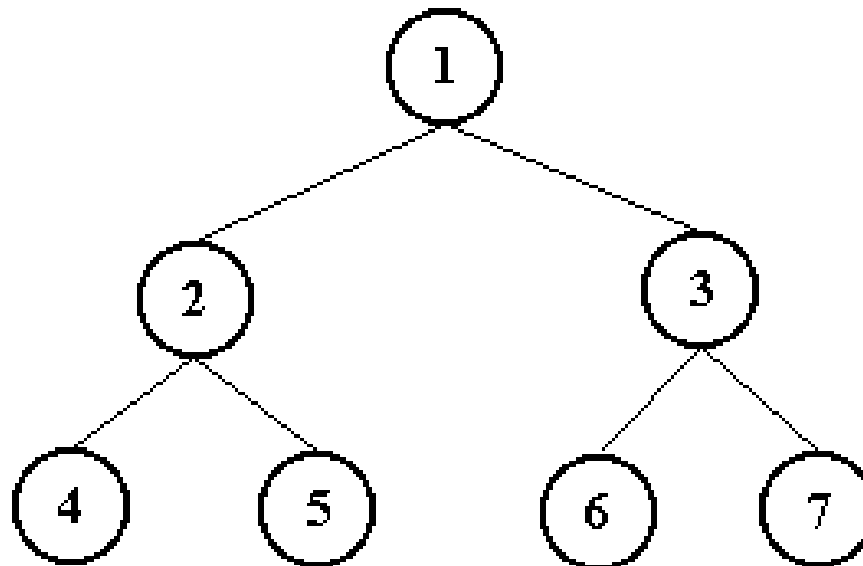
Uma árvore ordenada é aquela na qual os filhos de cada nó estão ordenados.
Assume-se ordenação da esquerda para a direita

Árvores

Definições

Árvore Cheia

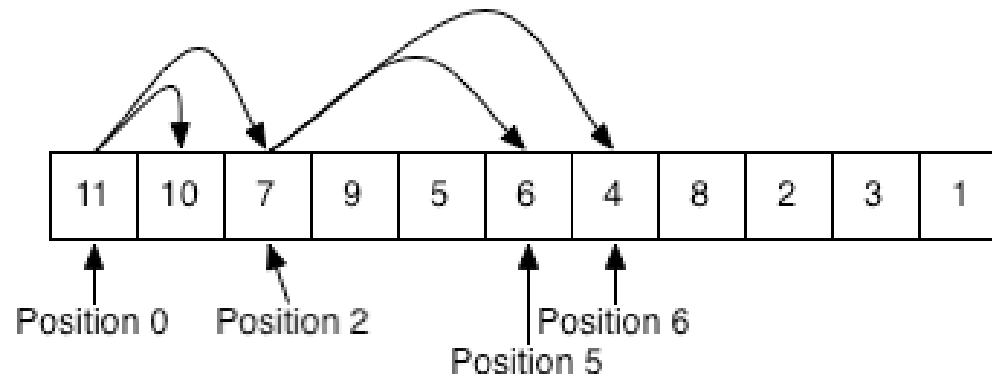
Uma árvore de grau d é uma árvore cheia se possui o número máximo de nós, isto é, todos os nós tem número máximo de filhos exceto as folhas, e todas as folhas estão na mesma altura.



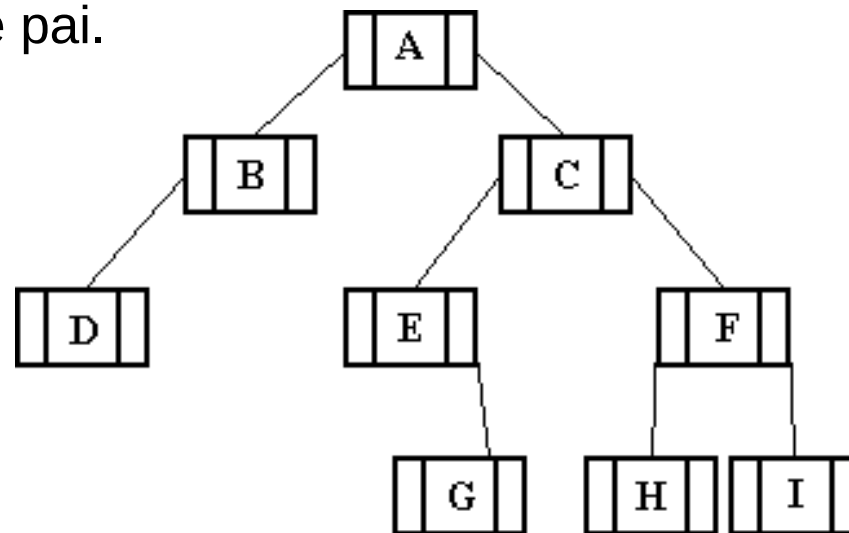
Árvores

Quanto a implementação, as árvores podem ser:

Estáticas: Os elementos são armazenados em um vetor.



Dinâmicas: Os elementos são alocados dinamicamente conforme necessidade. Cada elemento armazena os dados e um ponteiro para o filho da direita, esquerda e pai.



Árvores

Principais Estrutura de Dados do tipo Árvore:

Árvore Binária

Árvore AVL

Árvore B

Árvore B+

Árvore Rubro-Negra

Heaps *



UFABC

Universidade Federal do ABC
Centro de Matemática, Computação e Cognição

Árvores AVL

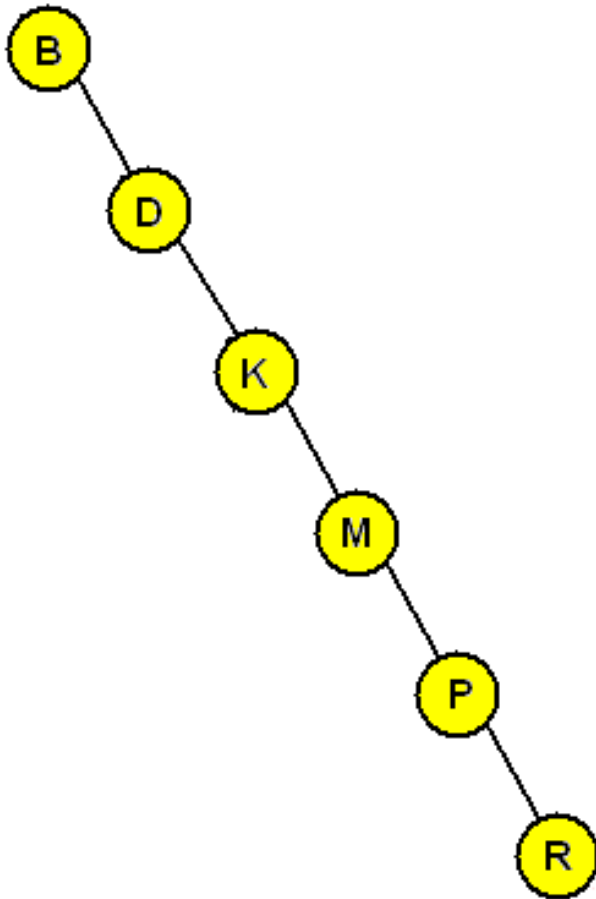
Monael Pinheiro Ribeiro, D.Sc.

Árvore Binária de Busca

- A forma de uma Árvore Binária de Busca depende da ordem em que seus elementos são inseridos.
- Elementos: B D K M P R

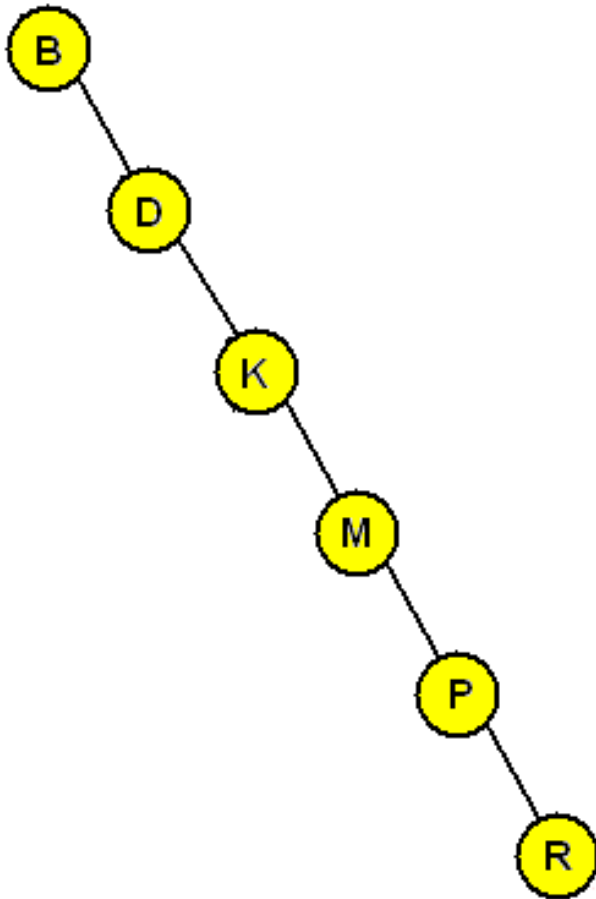
Árvore Binária de Busca

- A forma de uma Árvore Binária de Busca depende da ordem em que seus elementos são inseridos.
- Elementos: B D K M P R



Árvore Binária de Busca

- A forma de uma Árvore Binária de Busca depende da ordem em que seus elementos são inseridos.
- Elementos: B D K M P R



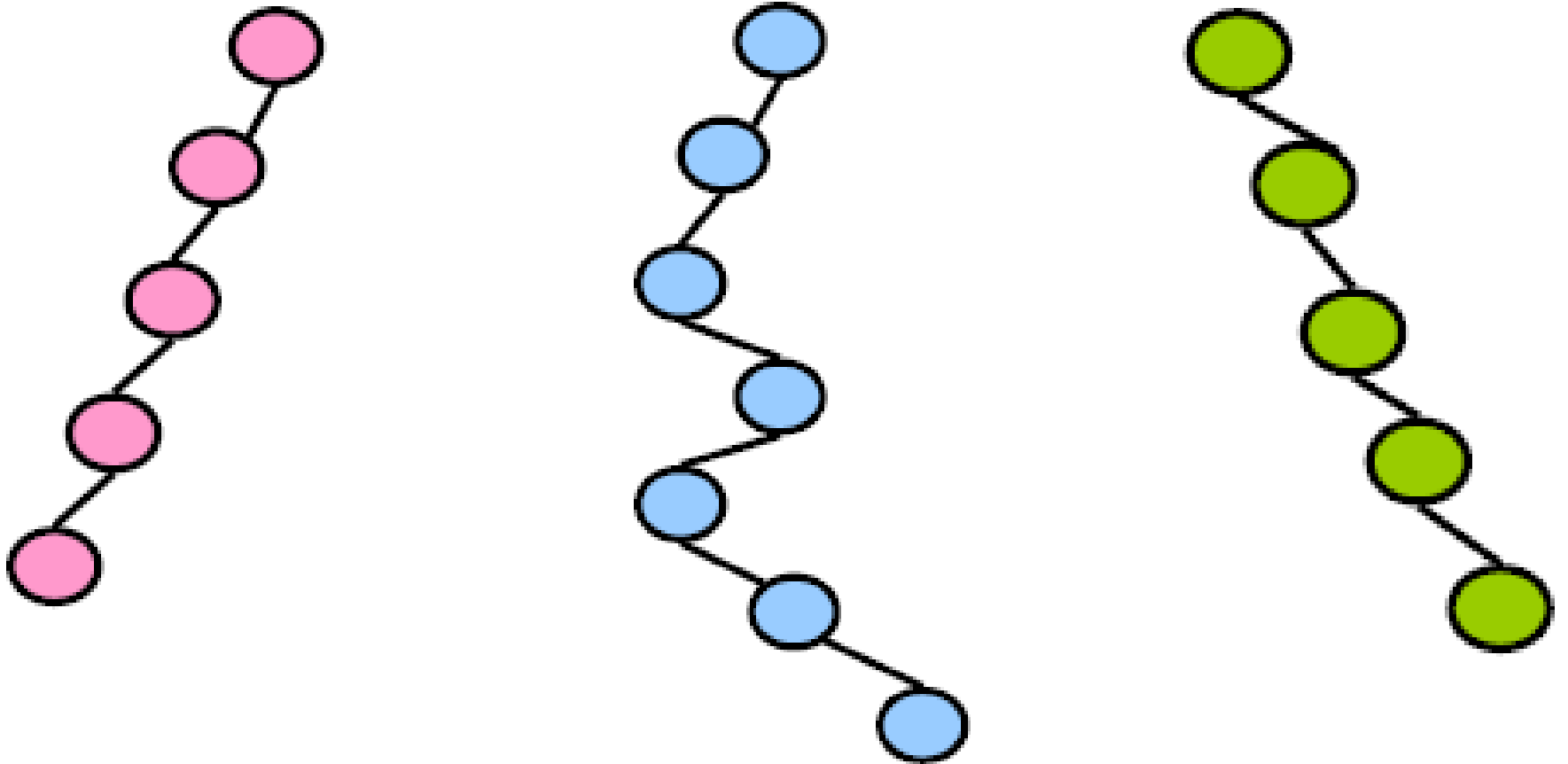
- Qual o custo para efetuar uma busca nesta Árvore ?

Árvores

Definições

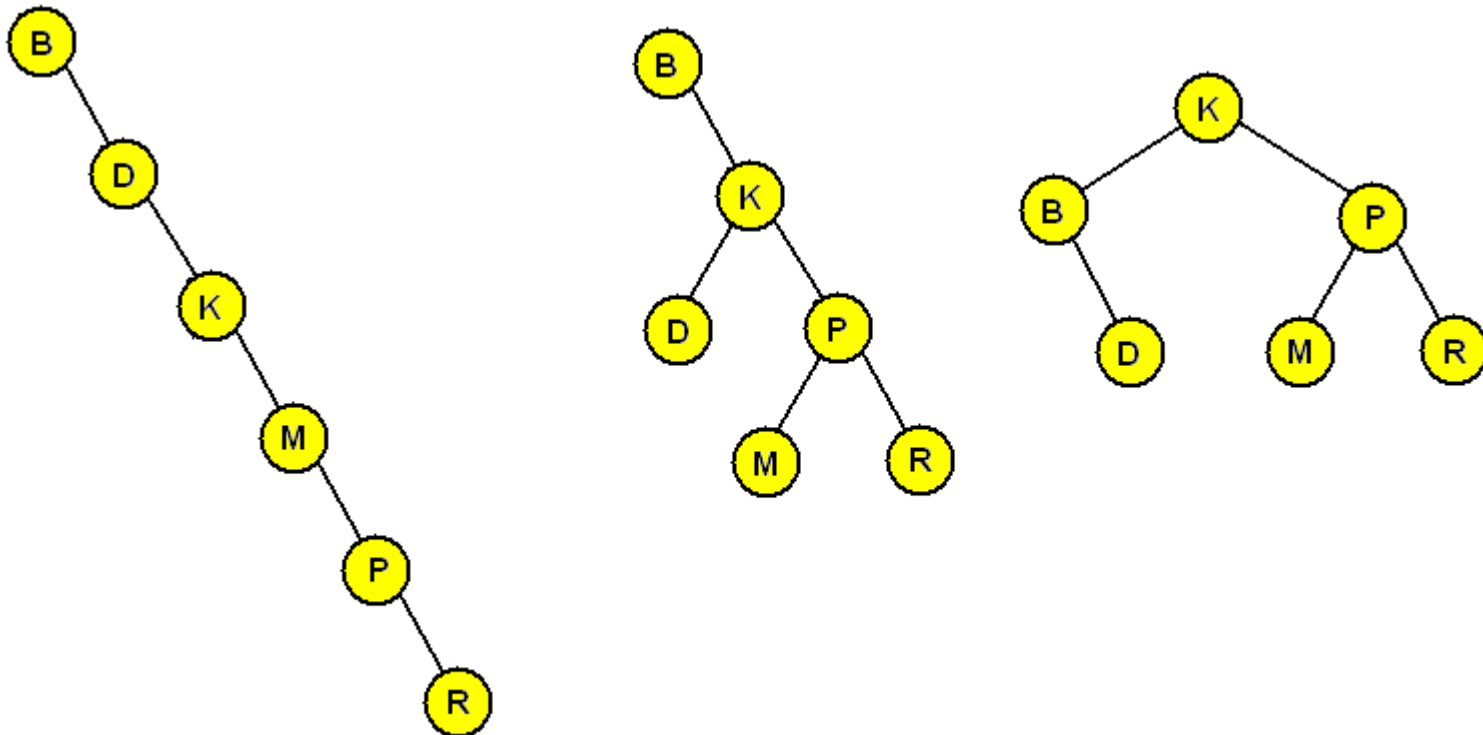
Árvore Zigue Zague

Trata-se de uma Árvore Binária de Busca com altura máxima. Onde todos os nós internos possuem exatamente um filho, com exceção da folha.



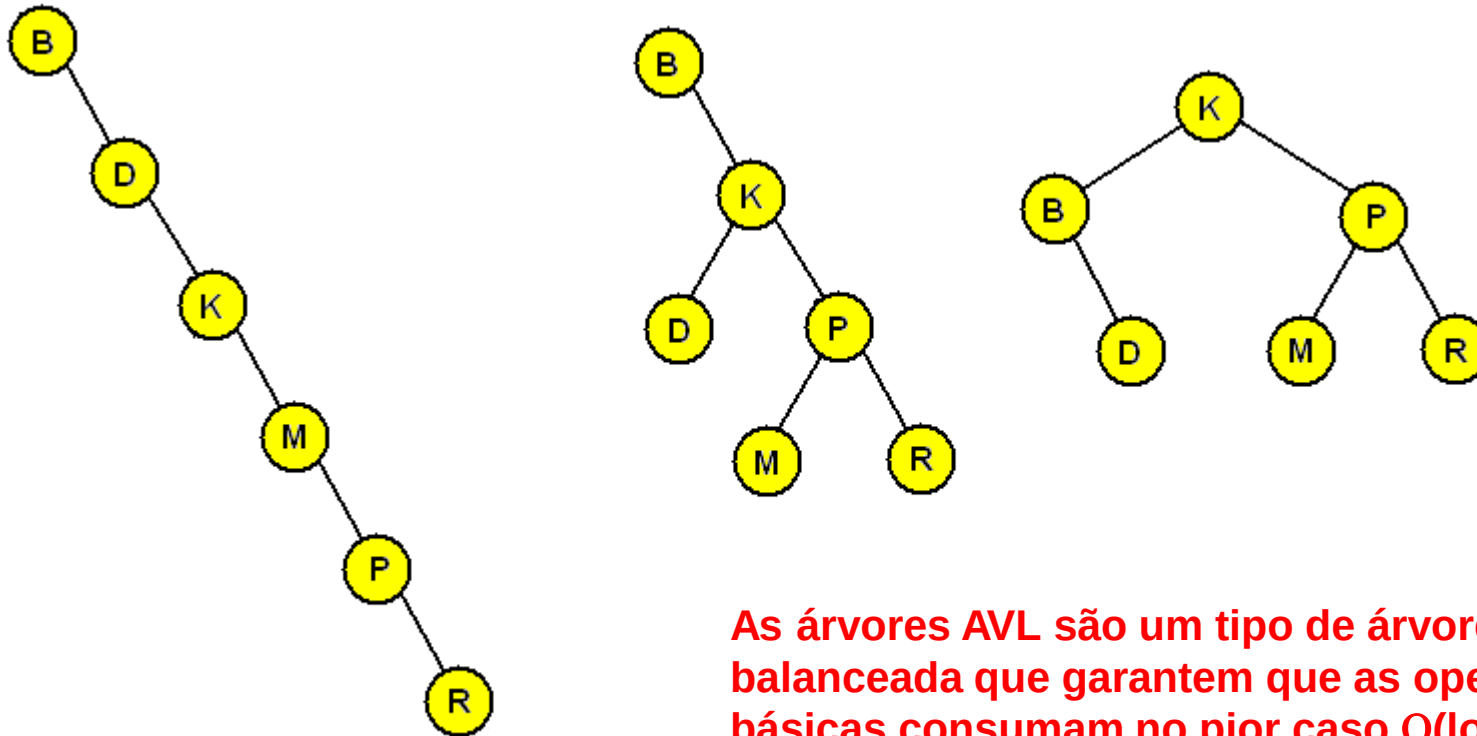
Árvore AVL

- Como vimos em nas árvores binárias, a vantagem de usar uma ED de árvore é justamente seu desempenho. Mas para garantir o bom desempenho dos algoritmos em árvores, deve-se garantir seu balanceamento



Árvore AVL

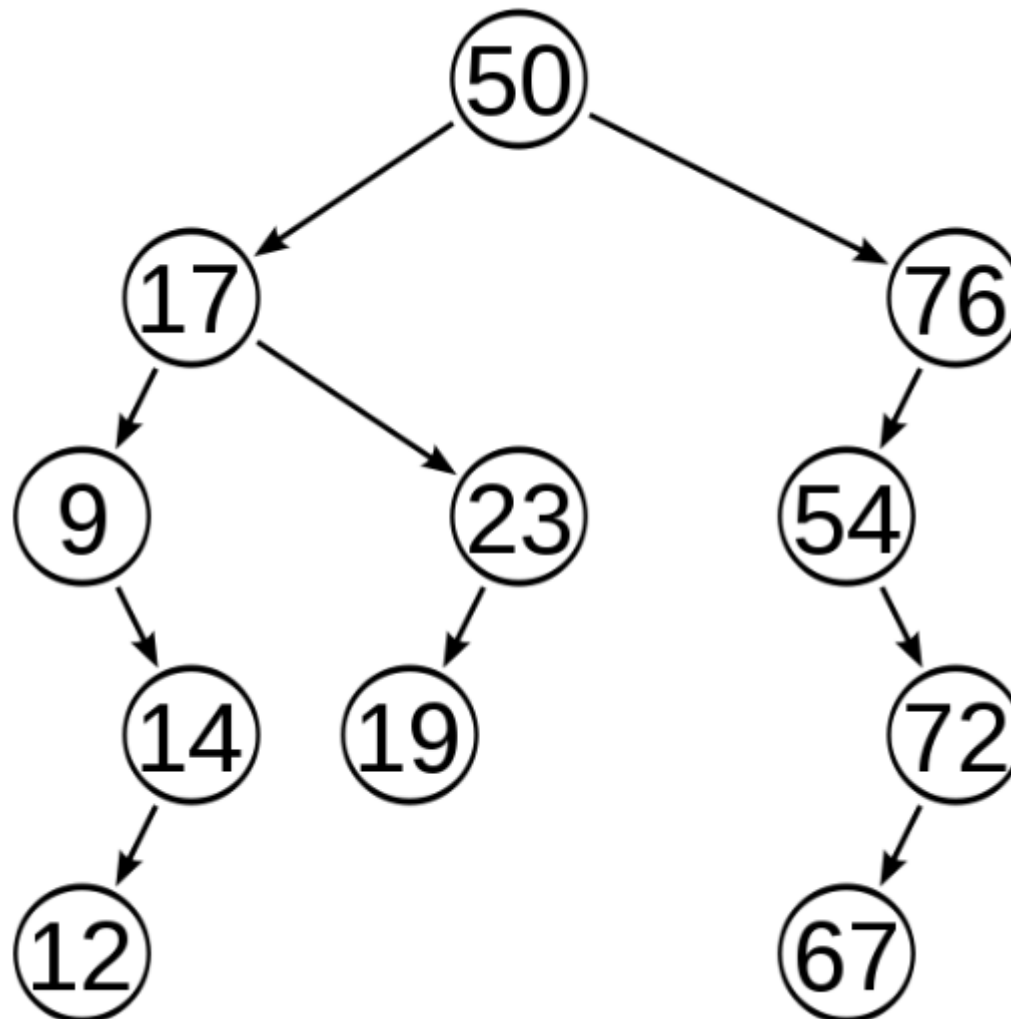
- Como vimos em nas árvores binárias, a vantagem de usar uma ED de árvore é justamente seu desempenho. Mas para garantir o bom desempenho dos algoritmos em árvores, deve-se garantir seu balanceamento



As árvores AVL são um tipo de árvore balanceada que garantem que as operações básicas consumam no pior caso $O(\log_2 n)$.

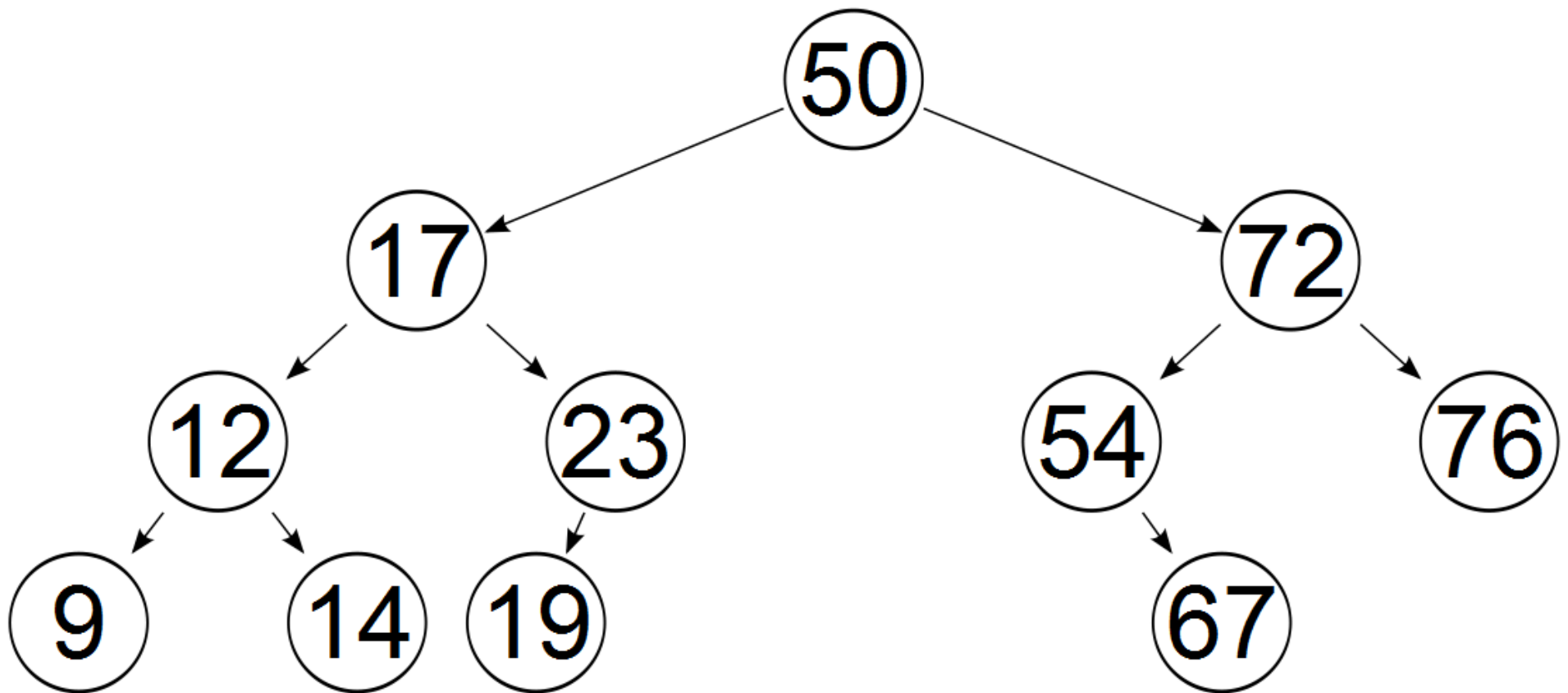
Árvore AVL

- Árvore Binária de Busca:



Árvore AVL

- Árvore AVL:

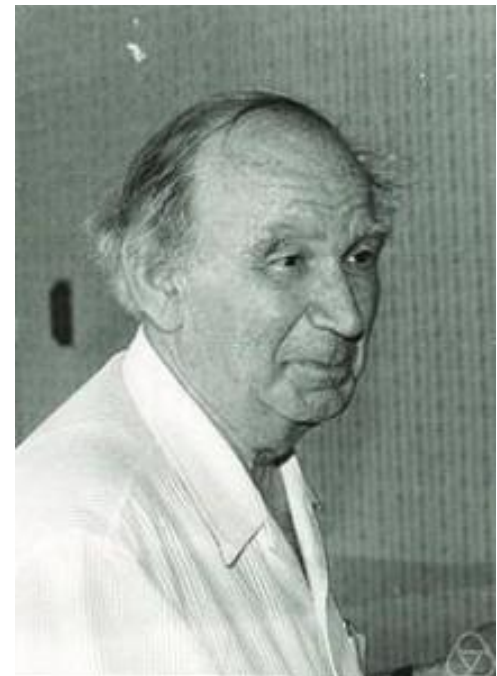


Árvore AVL

- São também conhecidas por “Árvores Balanceadas pela Altura”
- O nome vem de seus criadores:
 - Georgy Maximovich **A**ndelson-**V**elsky, e
 - Evgenii Mikhailovich **L**andis



☆ 08/01/1922 - † 26/04/2014



☆ 06/10/1921 - † 12/12/1997

Árvore AVL

- As árvores AVL possuem a seguinte propriedade:
 - Dado qualquer nó da árvore, a diferença entre a altura do seu ramo esquerdo e a altura do seu ramo direito é de no máximo de uma unidade.

Árvore AVL

- As árvores AVL possuem a seguinte propriedade:
 - Dado qualquer nó da árvore, a diferença entre a altura do seu ramo esquerdo e a altura do seu ramo direito é de no máximo de uma unidade.

As Árvores AVL são Árvores Binárias de Busca

Árvore AVL

Em Ciência da Computação uma árvore AVL é uma estrutura de dados como a árvore binária de busca que:

Consiste de uma seqüência de registros

Cada registro tem quatro campos:

uma campo para a chave.

um ponteiro para o filho da esquerda (subárvore da esquerda)

um ponteiro para o filho da direita (subárvore da direita)

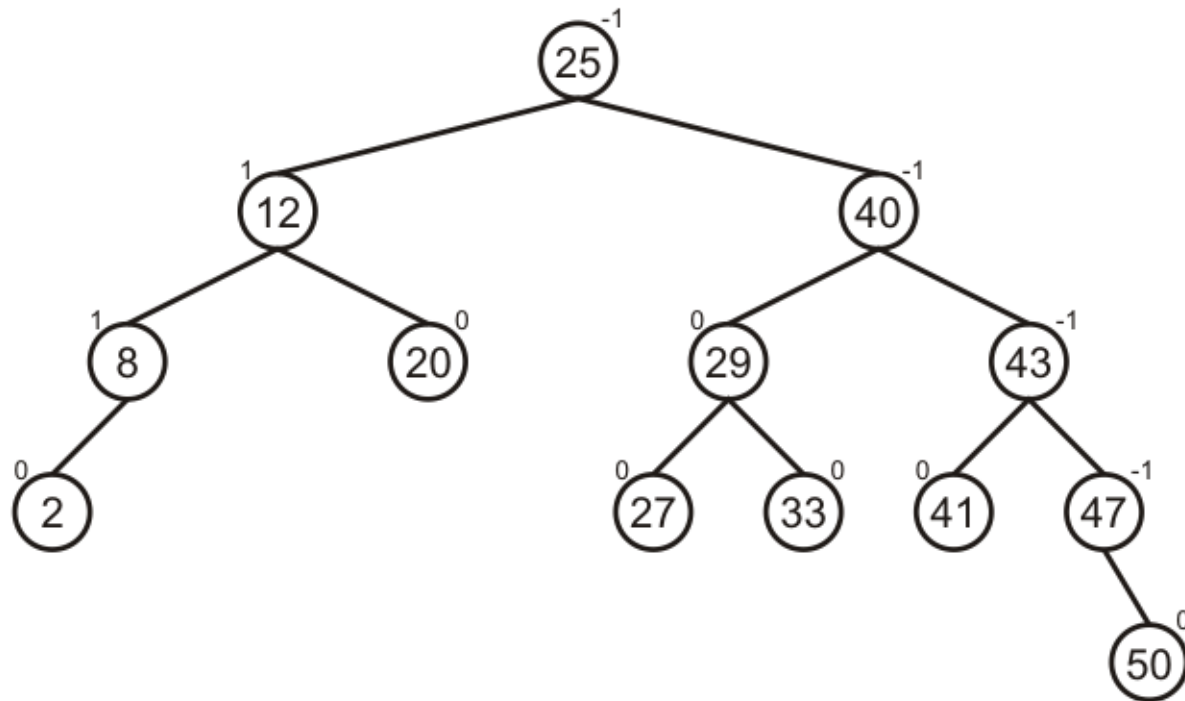
um ponteiro para o pai. (nó pai)

Ponteiro para raiz
Quantidade

pai		
esquerda	chave	direita

Árvore AVL

- Exemplo de representação de uma árvore AVL

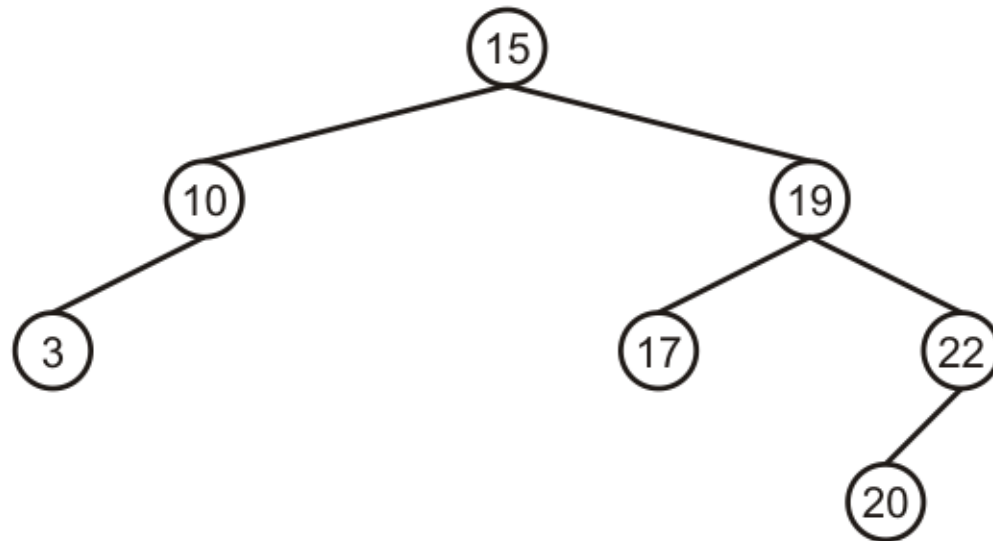


Árvore AVL

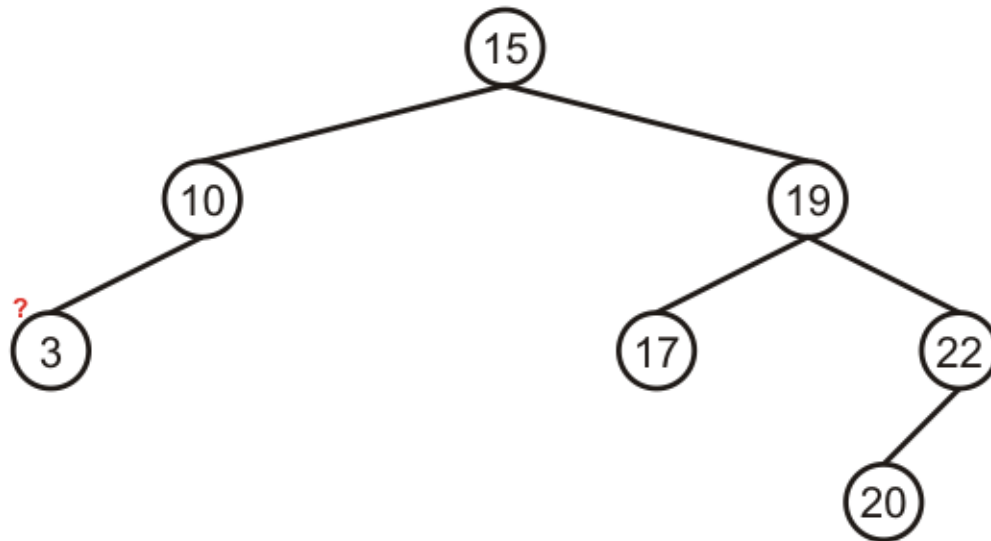
- Definição

- Uma Árvore Binária de Busca é uma Árvore AVL se, e somente se, para cada nó (incluindo a raiz) a altura do ramo esquerdo difere em no máximo uma unidade da altura do ramo direito.
- A diferença entre essas alturas é chamada de **fator de balanceamento** do nó.
- Este fator deve, então, ser -1, 0 ou 1 em todos os nós para que a árvore esteja balanceada.
- Se um nó é uma folha, seu fator de balanceamento é 0.

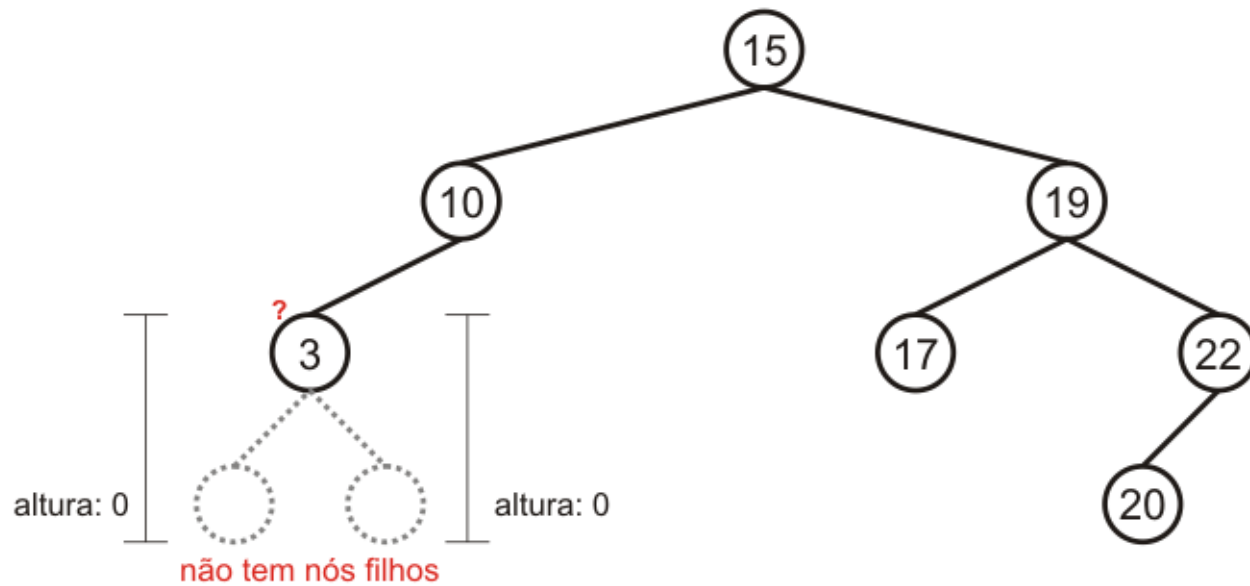
Fator de Balanceamento



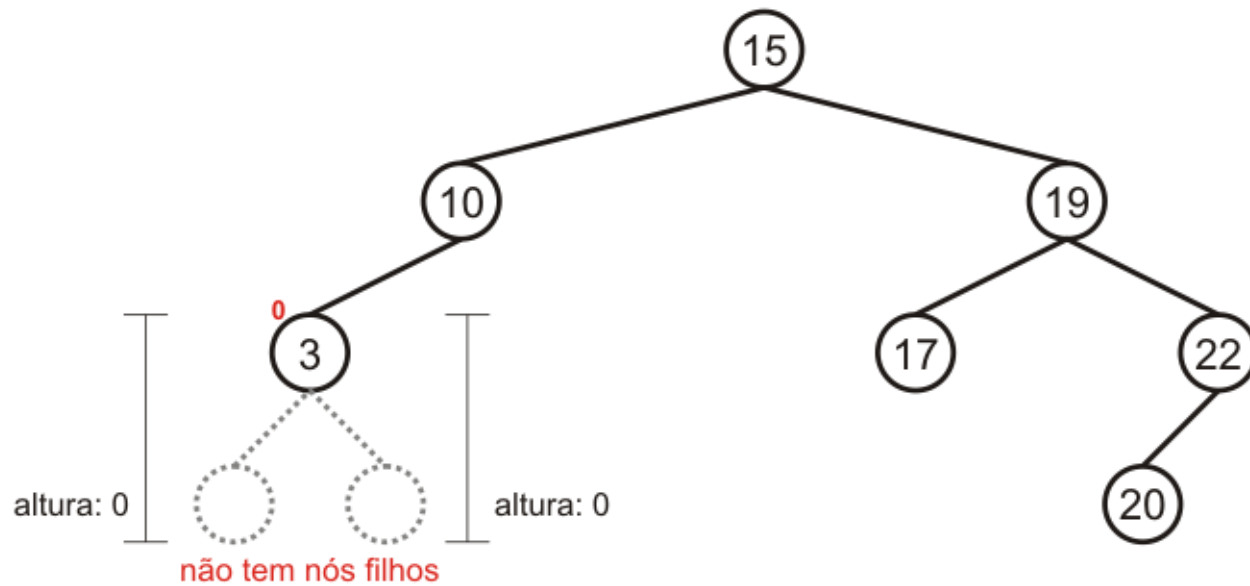
Fator de Balanceamento



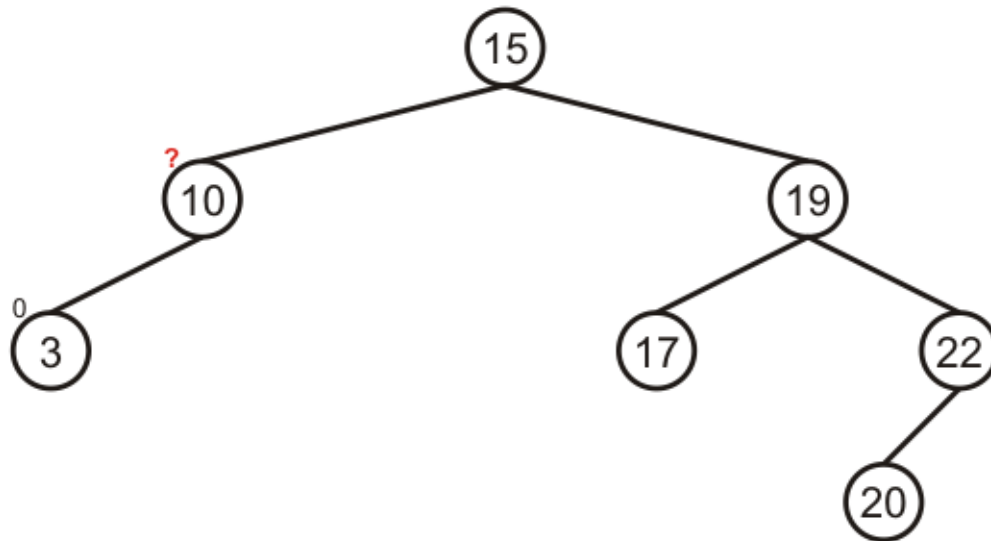
Fator de Balanceamento



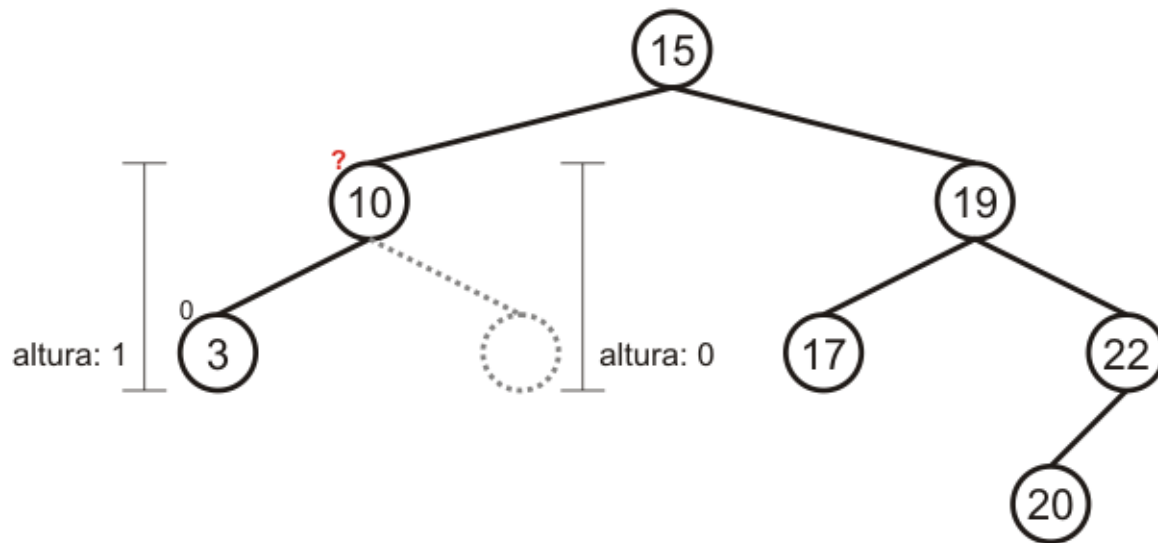
Fator de Balanceamento



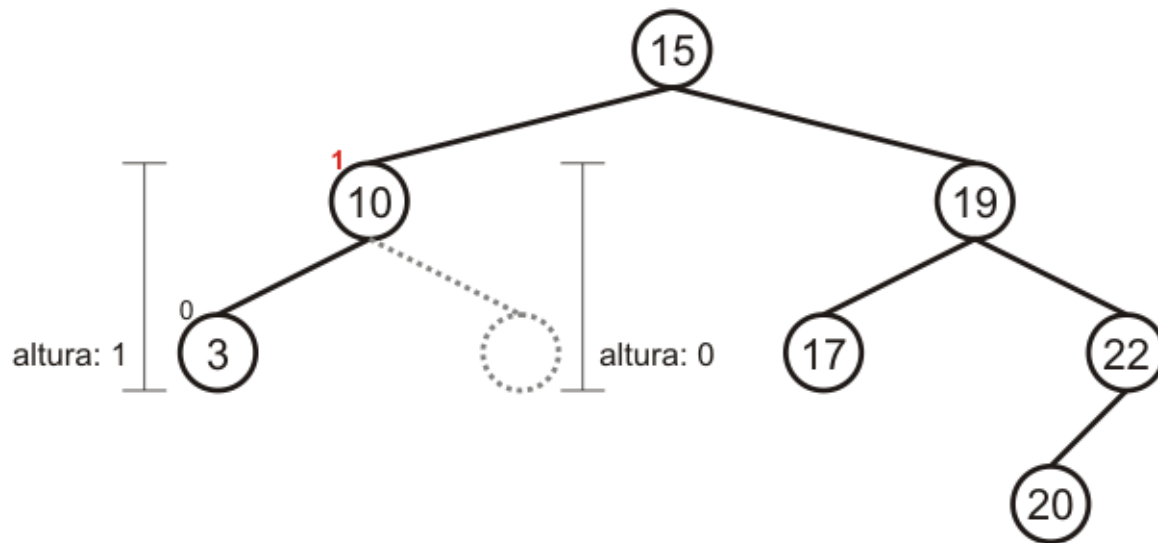
Fator de Balanceamento



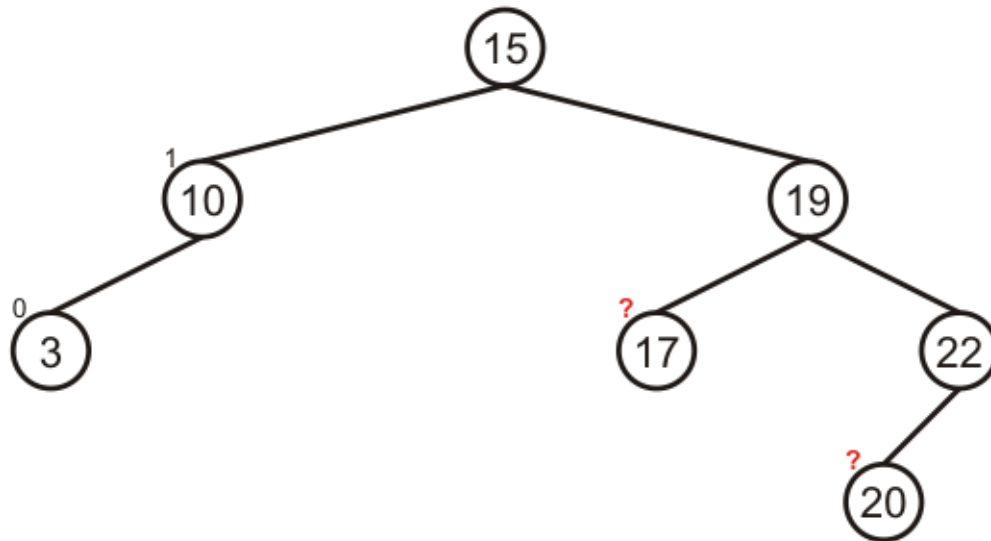
Fator de Balanceamento



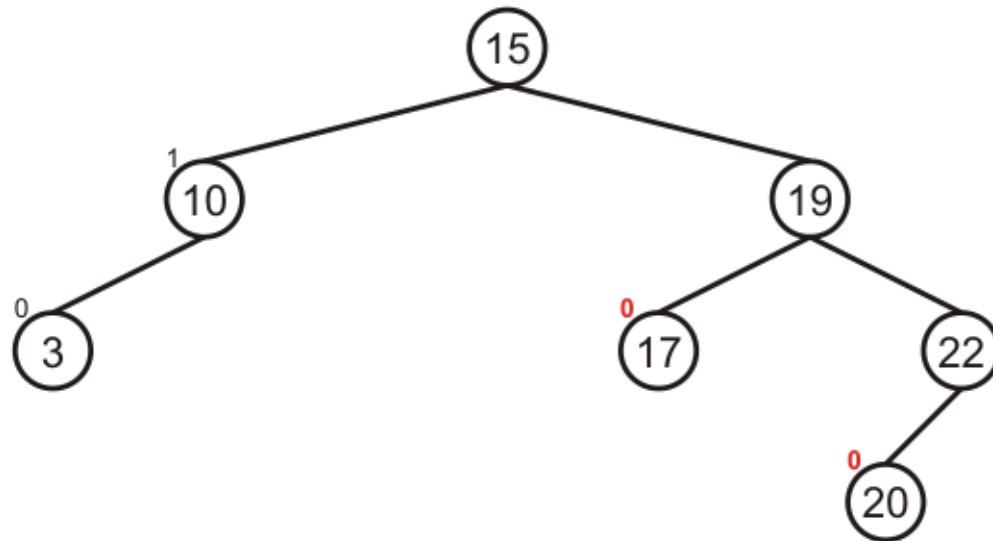
Fator de Balanceamento



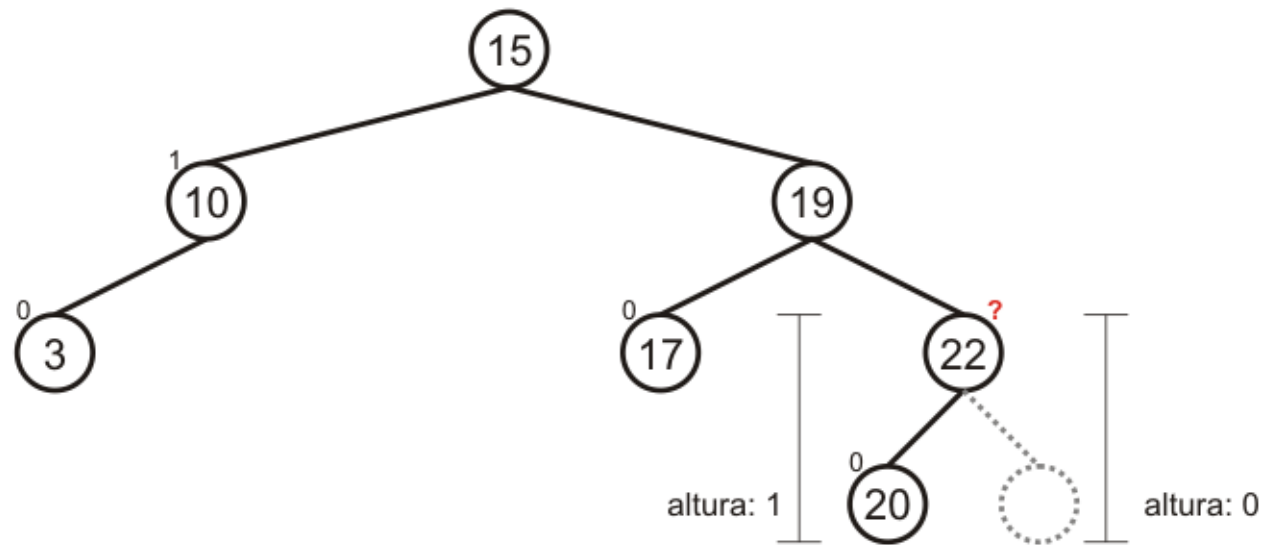
Fator de Balanceamento



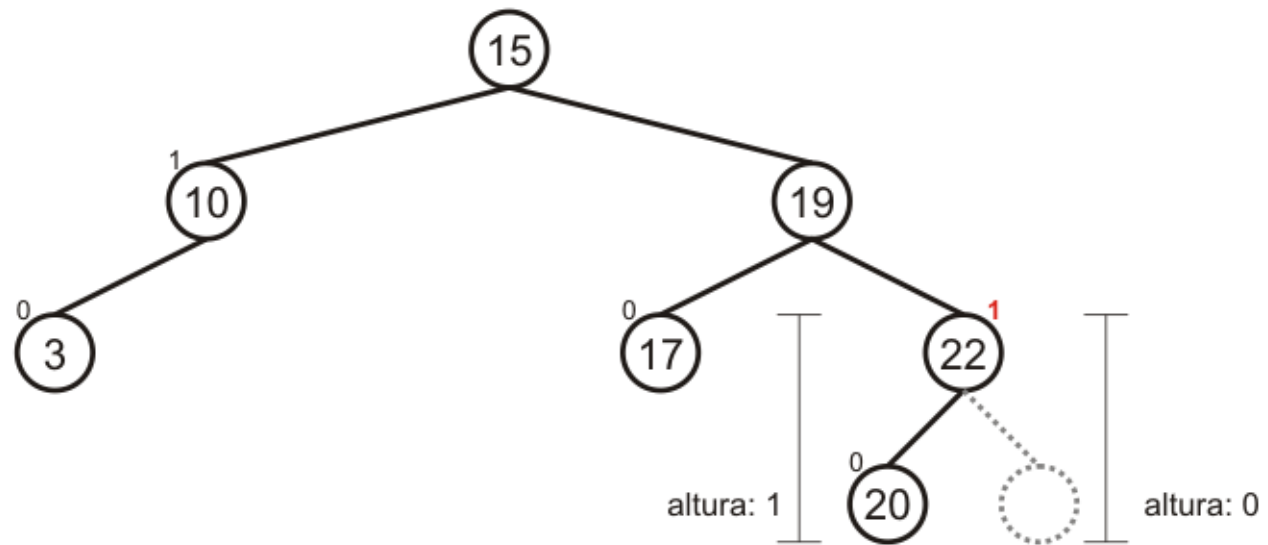
Fator de Balanceamento



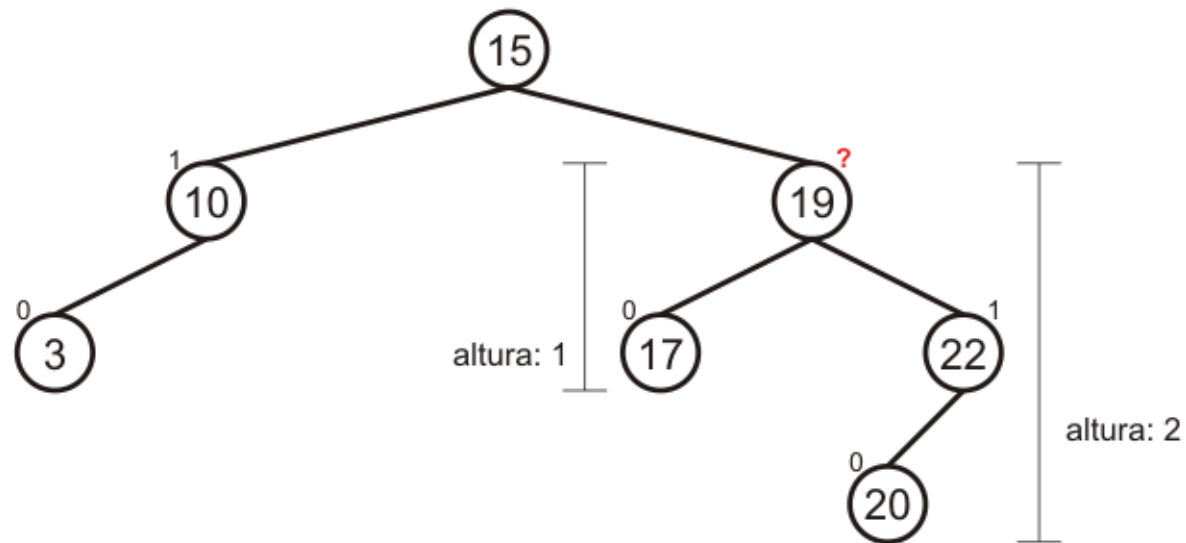
Fator de Balanceamento



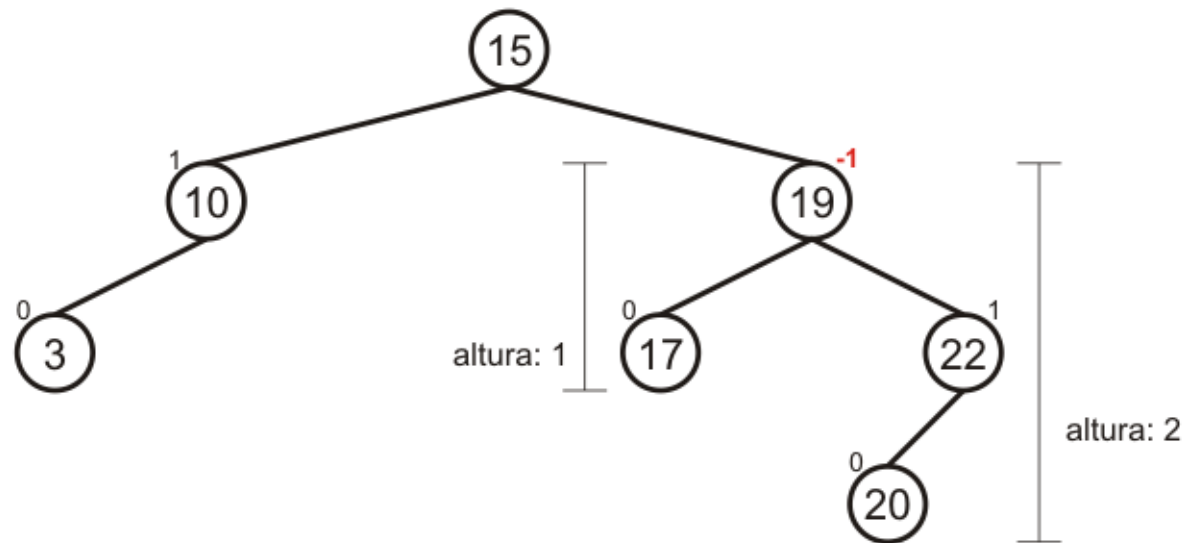
Fator de Balanceamento



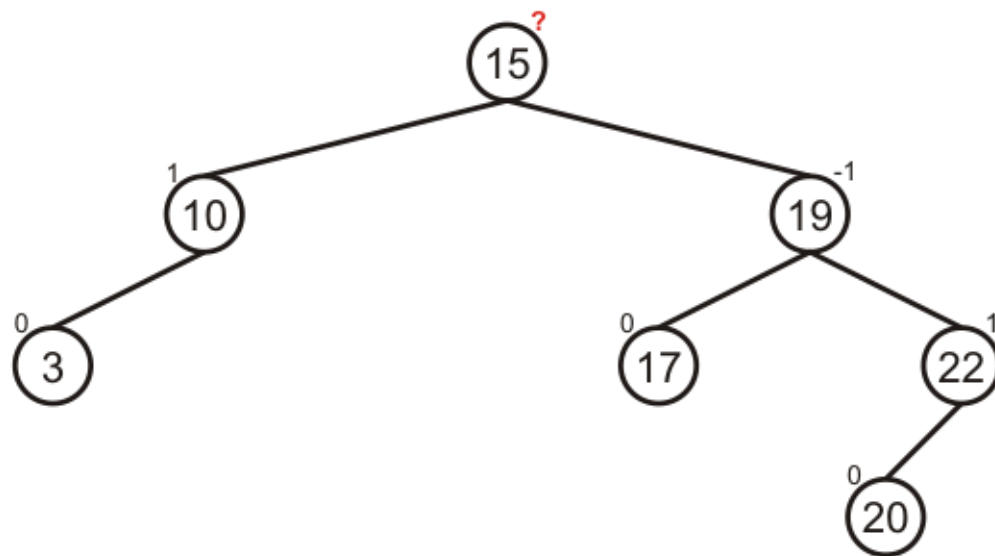
Fator de Balanceamento



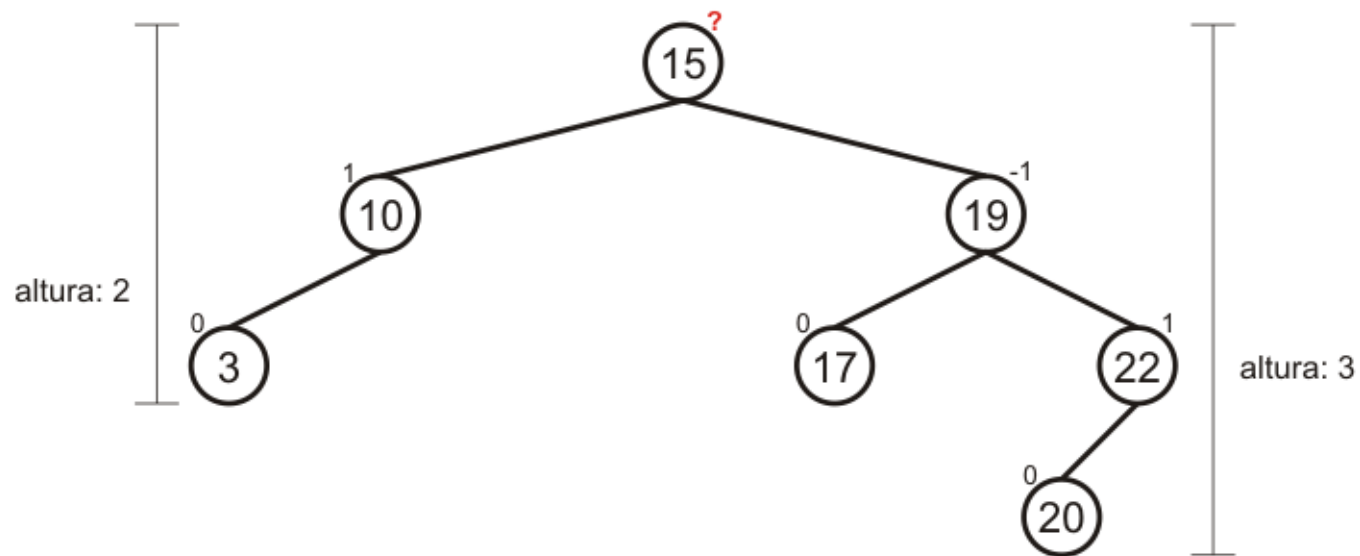
Fator de Balanceamento



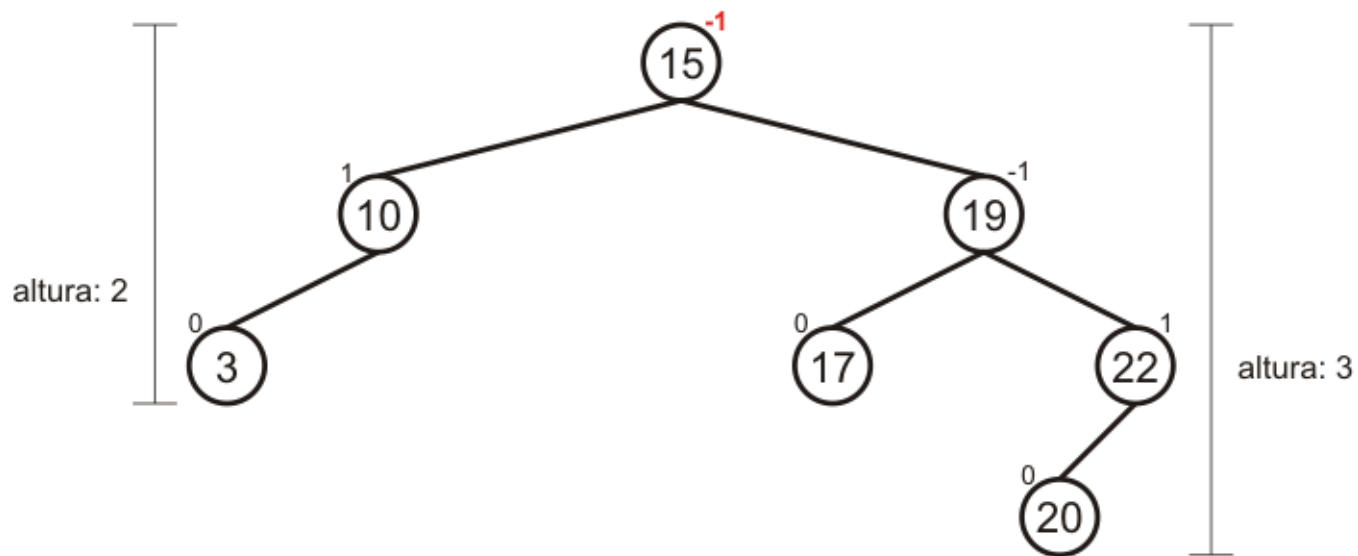
Fator de Balanceamento



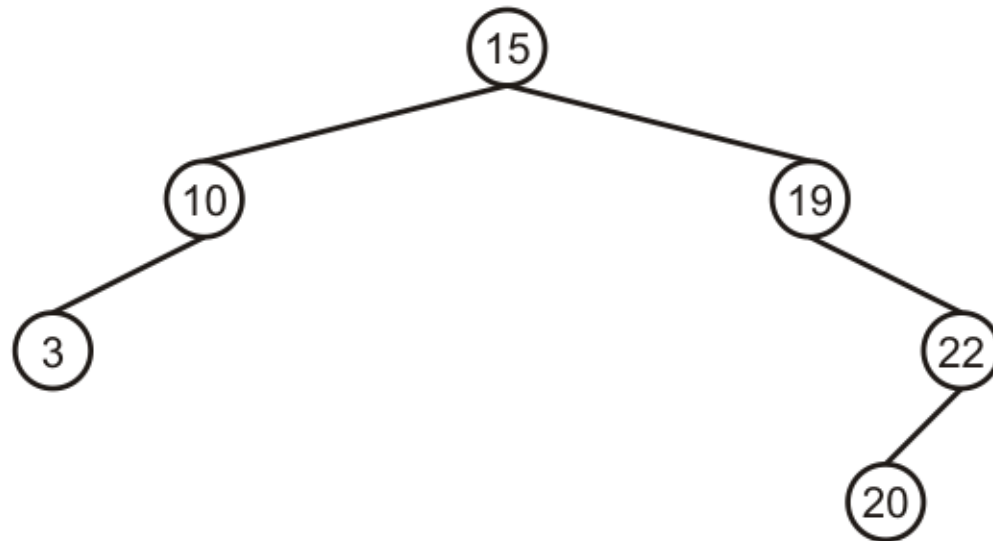
Fator de Balanceamento



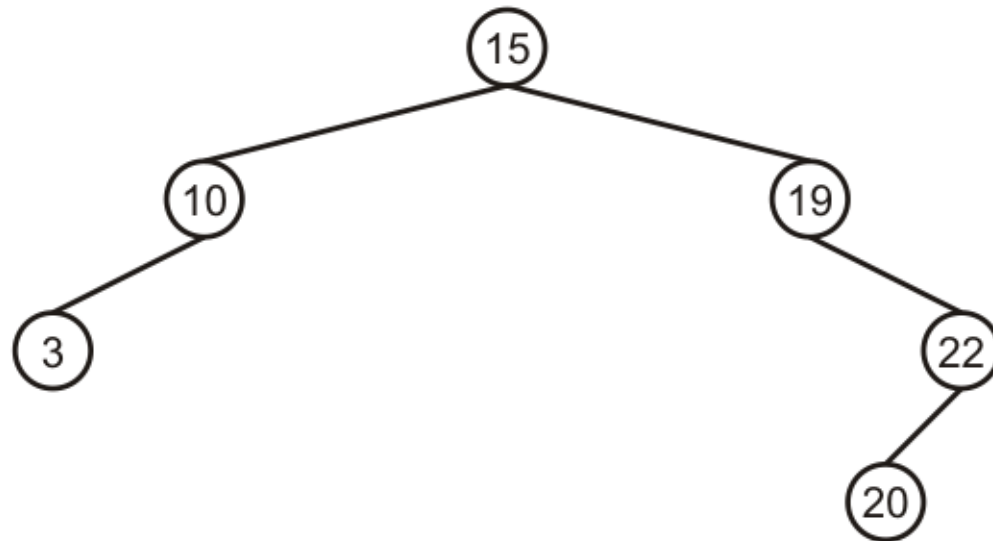
Fator de Balanceamento



Fator de Balanceamento

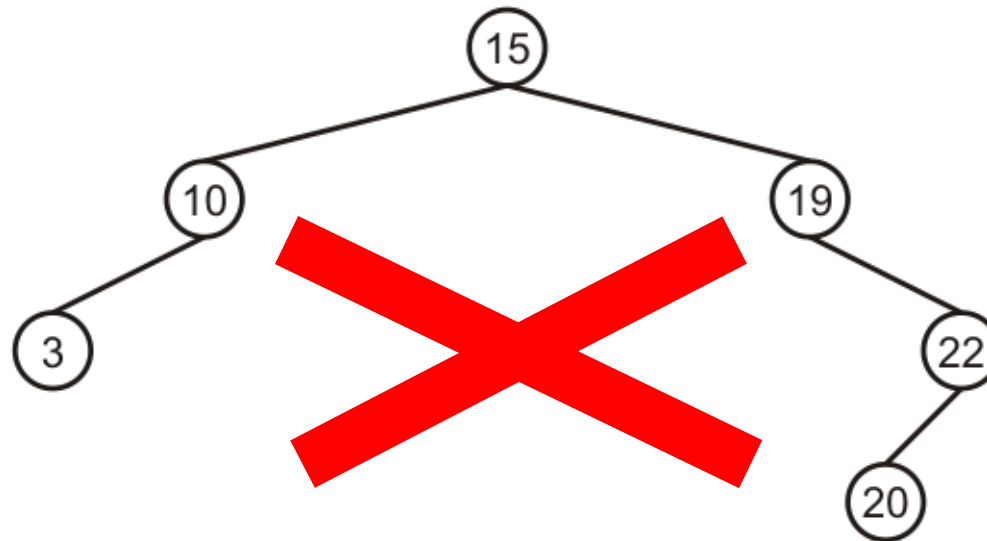


Fator de Balanceamento



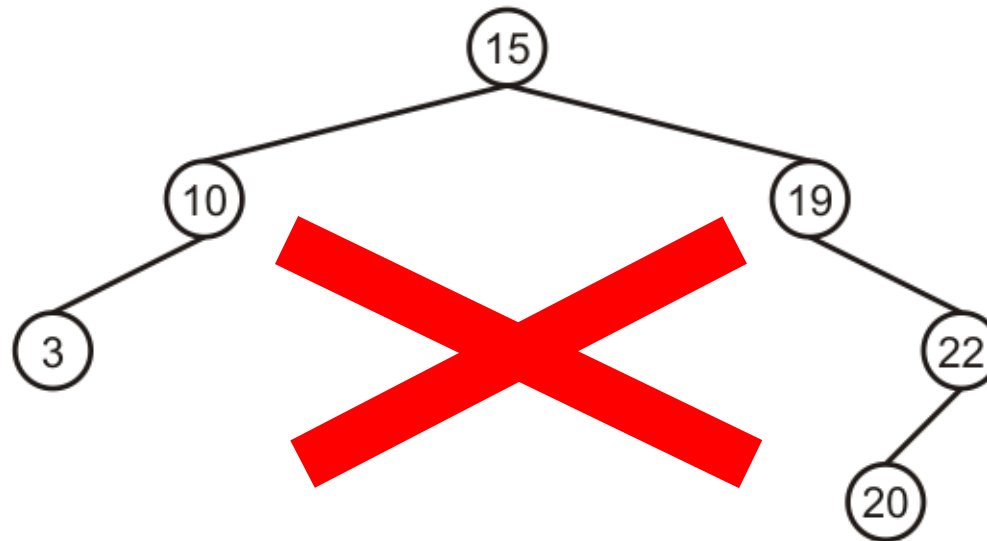
É uma árvore balanceada ?

Fator de Balanceamento



É uma árvore balanceada ?

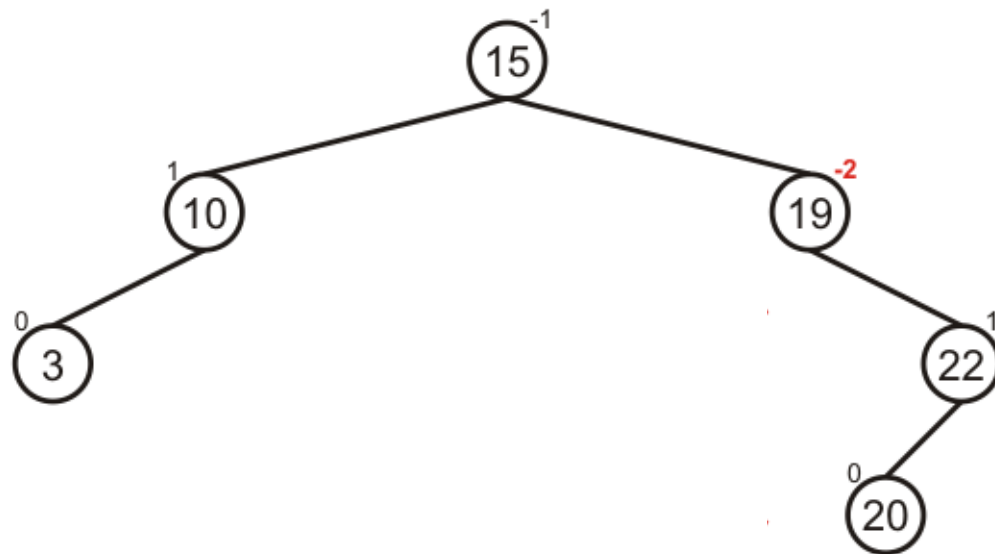
Fator de Balanceamento



É uma árvore balanceada ?

Por que?

Fator de Balanceamento



Árvore AVL

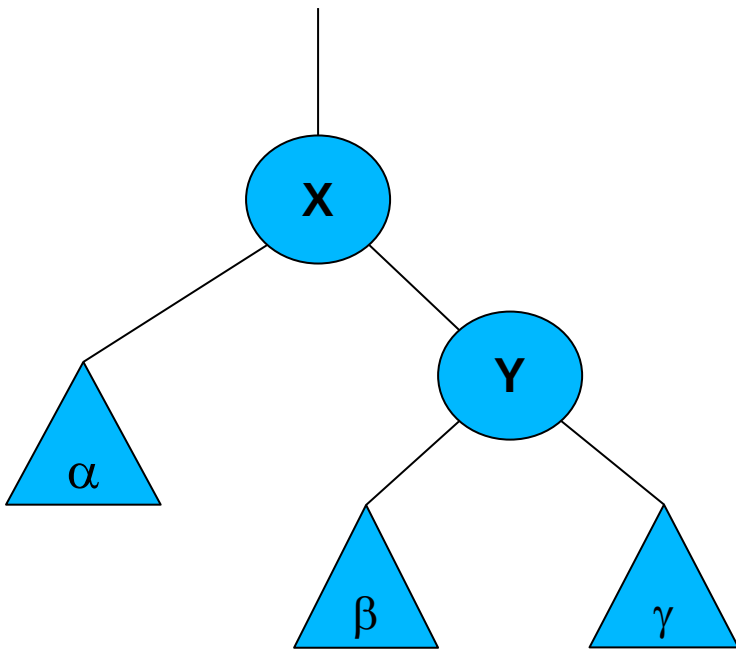
- A altura h de uma Árvore AVL de **n chaves** é de no máximo $1.44 \log_2(n + 1)$
- Esse resultado mostra a importância e utilidade de uma Árvore AVL, pois a busca, inserção e remoção tem complexidade de tempo $O(\log_2 n)$ no pior caso.
- Inserções e remoções feitas numa Árvore AVL podem modificar a sua estrutura. Deve-se garantir que o **balanceamento** seja mantido.
- Para isso, pode-se ter que mudar a estrutura da árvore em alguns de seus nós. A mudança da estrutura da árvore é feita por dois tipos de **rotações** em ramos da árvore: **left-rotate** e **right-rotate**.

Rotações

- As rotações são funções executadas quando se insere e remove nós de uma árvore balanceada, como a Árvore AVL, com o objetivo de manter o balanceamento.

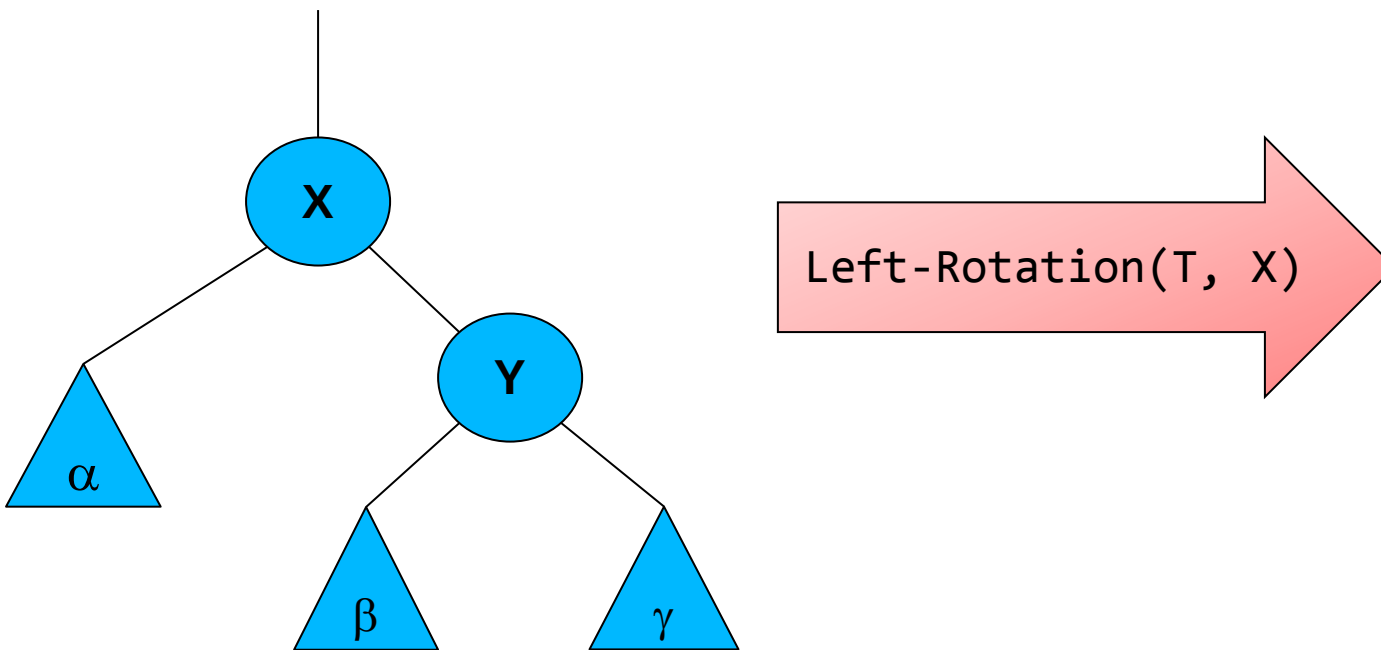
Rotações

- As rotações são funções executadas quando se insere e remove nós de uma árvore balanceada, como a Árvore AVL, com o objetivo de manter o balanceamento.
- **Rotação para Esquerda:**



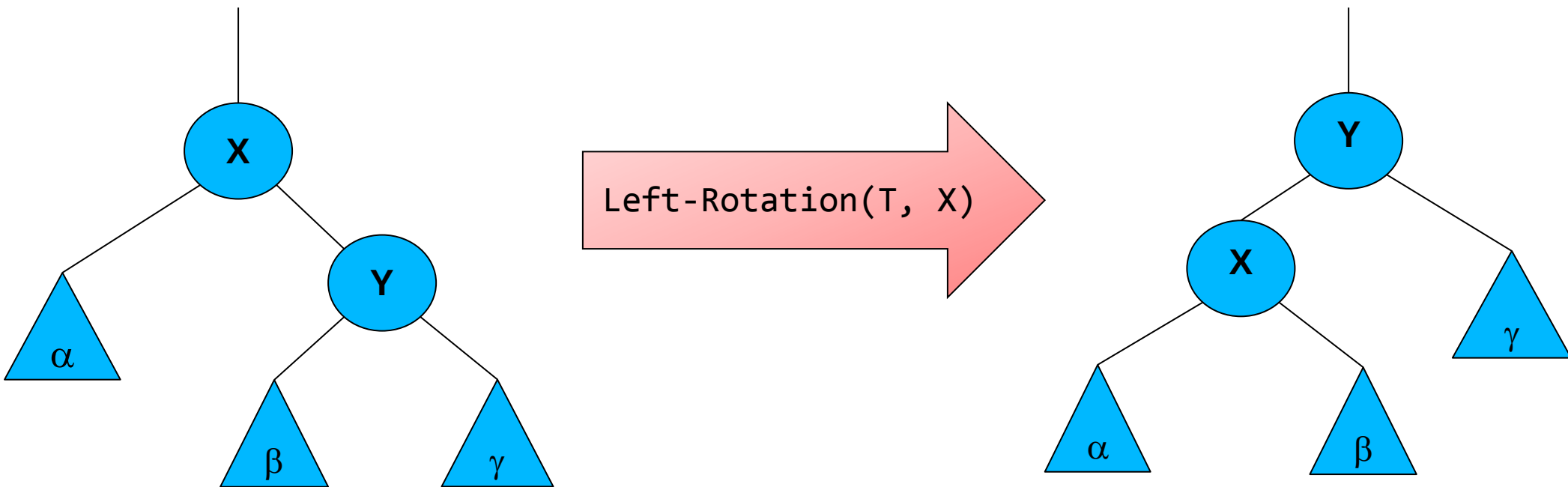
Rotações

- As rotações são funções executadas quando se insere e remove nós de uma árvore balanceada, como a Árvore AVL, com o objetivo de manter o balanceamento.
- **Rotação para Esquerda:**



Rotações

- As rotações são funções executadas quando se insere e remove nós de uma árvore balanceada, como a Árvore AVL, com o objetivo de manter o balanceamento.
- **Rotação para Esquerda:**

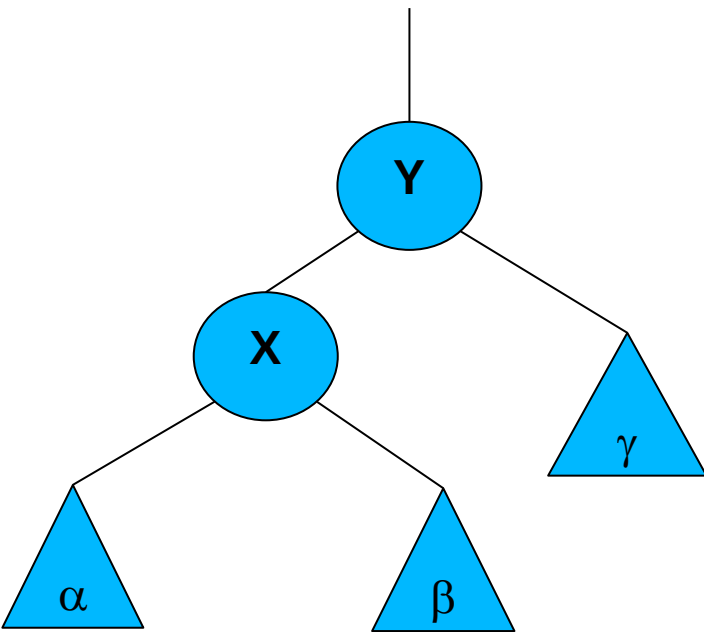


Rotações

- Rotação para Esquerda
 - Passos:
 - Define-se x e y .
 - Subárvore da esquerda de y passa a ser subárvore da direita de x .
 - O pai de x passa a ser pai de y .
 - Se x não possuir pai, então y será raiz da árvore.
 - senão, se x for filho direito, y passa a ser filho direito do pai de x .
 - senão, y passa a ser filho esquerdo do pai de x .
 - x passa a ser filho da esquerda de y .

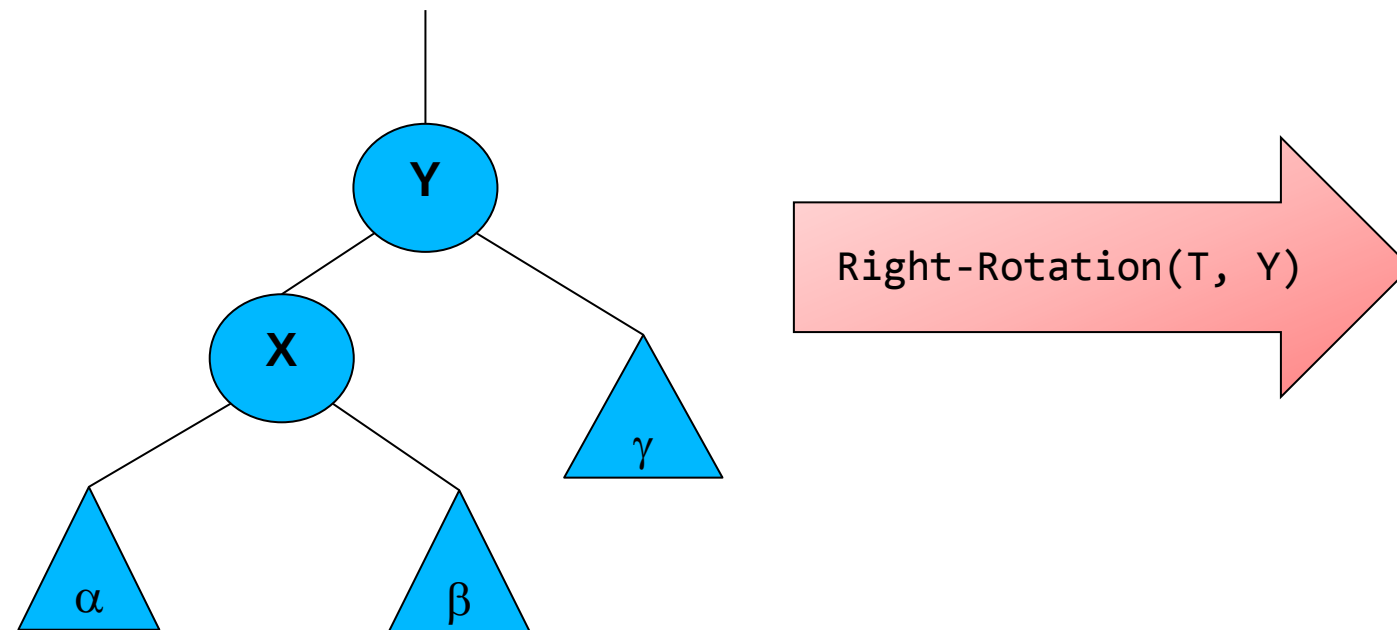
Rotações

- As rotações são funções executadas quando se insere e remove nós de uma árvore balanceada, como a Árvore AVL, com o objetivo de manter o balanceamento.
- **Rotação para Direita:**



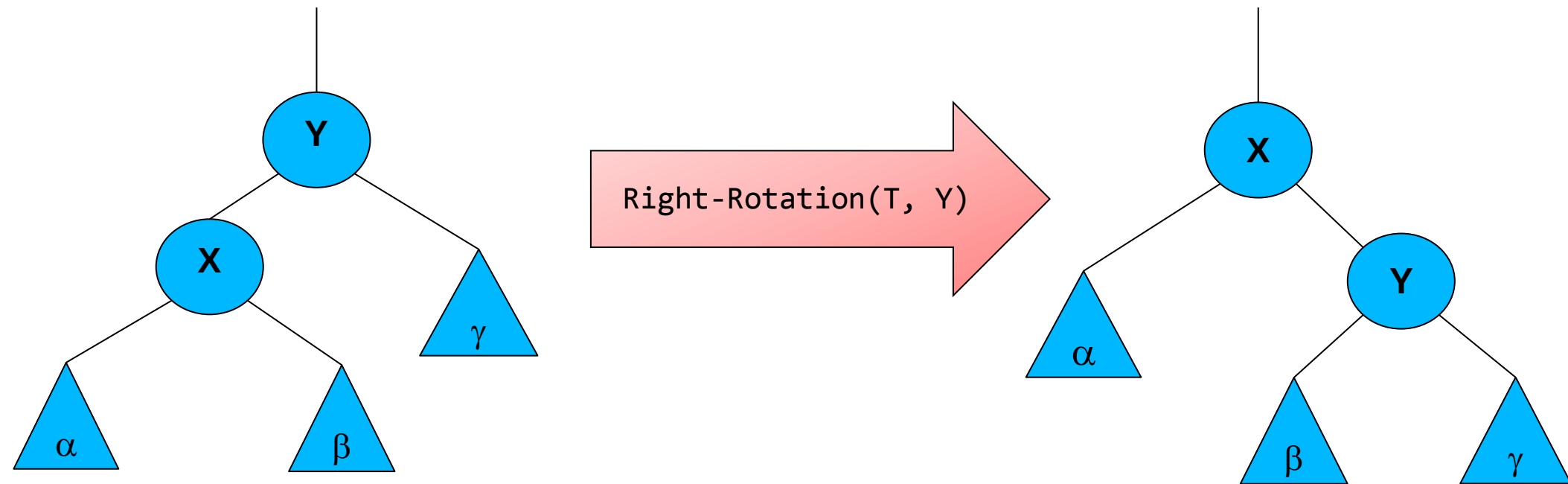
Rotações

- As rotações são funções executadas quando se insere e remove nós de uma árvore balanceada, como a Árvore AVL, com o objetivo de manter o balanceamento.
- **Rotação para Direita:**



Rotações

- As rotações são funções executadas quando se insere e remove nós de uma árvore balanceada, como a Árvore AVL, com o objetivo de manter o balanceamento.
- **Rotação para Direita:**



Rotações

- Rotação para Direita
 - Passos:
 - Define-se x e y .
 - Subárvore da direita de x passa a ser subárvore da esquerda de y .
 - O pai de y passa a ser pai de x .
 - Se y não possuir pai, então x será raiz da árvore.
 - senão, se y for filho direito, x passa a ser filho direito do pai de y .
 - senão, x passa a ser filho esquerdo do pai de y .
 - y passa a ser filho da direita de x .

Árvore AVL

- Inserção

- A inserção de um elemento em uma Árvore AVL pode ser realizada em tempo $O(\log_2 n)$
- Na realidade, usa-se uma versão modificada da inserção em uma árvore binária de busca.
- A seguir, para garantir o balanceamento da Árvore AVL, invoca-se um procedimento auxiliar que verifica os nós e promove rotações.

Lembrando a inserção em uma Árvore Binária de Busca

- **Inserir**

- **Caso particular (árvore vazia):**

- Aloque espaço para o item e atribua-lhe os dados;
 - Faça com que o campo raiz da árvore aponte para o novo item.

- **Caso geral (se houver itens na árvore):**

- Percorrer a partir da raiz todos os itens, checando:
 - Se o campo chave do item for maior que a chave do novo item:
 - Vá para o filho esquerdo do item.
 - Se o campo chave do item for menor que a chave do novo item:
 - Vá para o filho direito do item.
 - Até encontrar um item nulo. Ao encontrar:
 - Se o campo chave do pai for maior que a chave do novo item:
 - Faça campo esquerda do pai apontar para o novo item.
 - Se o campo chave do pai for menor que a chave do novo item:
 - Faça campo direita do pai apontar para o novo item.
 - Faça campo pai do novo item apontar para o pai.

- Incremente o campo quantidade da árvore em uma unidade.

Árvore AVL

- Inserção

- No caso da inserção em uma árvore AVL, acrescentar ao final a seguinte instrução:
 - Invocar o procedimento para correção do balanceamento eventualmente quebrado com a inserção.

Inserção em uma Árvore AVL

- **Inserir**

- **Caso particular (árvore vazia):**

- Aloque espaço para o item e atribua-lhe os dados;
- Faça com que o campo raiz da árvore aponte para o novo item.

- **Caso geral (se houver itens na árvore):**

- Percorrer a partir da raiz todos os itens, checando:
 - Se o campo chave do item for maior que a chave do novo item:
 - Vá para o filho esquerdo do item.
 - Se o campo chave do item for menor que a chave do novo item:
 - Vá para o filho direito do item.
- Até encontrar um item nulo. Ao encontrar:
 - Se o campo chave do pai for maior que a chave do novo item:
 - Faça campo esquerda do pai apontar para o novo item.
 - Se o campo chave do pai for menor que a chave do novo item:
 - Faça campo direita do pai apontar para o novo item.
 - Faça campo pai do novo item apontar para o pai.
- Incremente o campo quantidade da árvore em uma unidade.
- **Invocar o procedimento de correção de balanceamento da Árvore AVL.**

Árvore AVL

- Remoção

- A remoção de um elemento em uma Árvore AVL pode ser realizada também em tempo $O(\log_2 n)$
- Na realidade, usa-se uma versão modificada da remoção em uma árvore binária de busca.
- E, para garantir o balanceamento da Árvore AVL, invoca-se um procedimento auxiliar que verifica os nós e promove rotações.

Lembrando a remoção em uma Árvore Binária de Busca

- **Remover**

- **Caso 1: removendo um nó folha:**

- Localize o nó a ser removido. Se for um nó folha, então:
 - Se não for raiz:
 - Faça o pai do nó a ser removido apontar para NULL.
 - Se for raiz:
 - Faça o campo raiz da árvore apontar para NULL.

- **Caso 2: removendo um nó com 1 filho:**

- Localize o nó a ser removido. Se o nó tiver apenas 1 filho:
 - Se não for raiz:
 - Faça o pai do nó a ser removido apontar para o neto.
 - Se for raiz:
 - Faça o campo raiz da árvore apontar para o neto.

- **Caso 3: removendo um nó com 2 filhos:**

- Localize o nó a ser removido. Se o nó tiver 2 filhos:
 - Busque o antecessor ou sucessor;
 - Se o sucessor/antecessor tiver filhos, aplique a remoção recursivamente sobre a subárvore do sucessor/antecessor;
 - Substitua o nó a ser removido pelo antecessor ou sucessor.

Árvore AVL

- Remoção

- No caso da remoção em uma árvore AVL, acrescentar ao final a seguinte instrução:
 - Invocar o procedimento para correção do balanceamento eventualmente quebrado com a remoção.

Remoção em uma Árvore AVL

- **Remove**

- **Caso 1: removendo um nó folha:**

- Localize o nó a ser removido. Se for um nó folha, então:
 - Se não for raiz:
 - Faça o pai do nó a ser removido apontar para NULL.
 - Se for raiz:
 - Faça o campo raiz da árvore apontar para NULL.

- **Caso 2: removendo um nó com 1 filho:**

- Localize o nó a ser removido. Se o nó tiver apenas 1 filho:
 - Se não for raiz:
 - Faça o pai do nó a ser removido apontar para o neto.
 - Se for raiz:
 - Faça o campo raiz da árvore apontar para o neto.

- **Caso 3: removendo um nó com 2 filhos:**

- Localize o nó a ser removido. Se o nó tiver 2 filhos:
 - Busque o antecessor ou sucessor;
 - Se o sucessor/antecessor tiver filhos, aplique a remoção recursivamente sobre a subárvore do sucessor/antecessor;
 - Substitua o nó a ser removido pelo antecessor ou sucessor.

- **Invocar o procedimento de correção de balanceamento da Árvore AVL.**

Correção de balanceamento em uma Árvore AVL

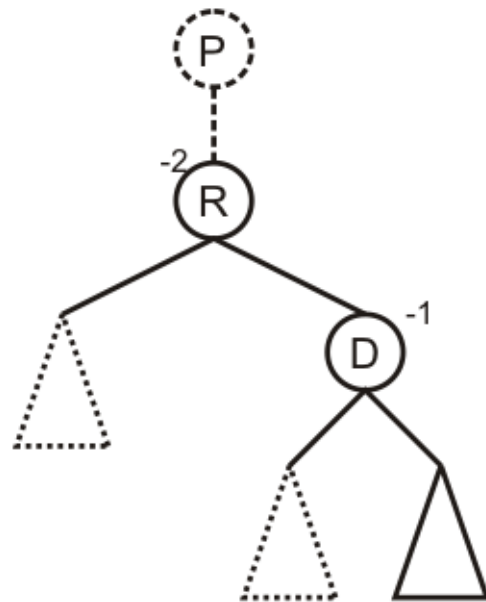
- A inserção e a remoção de nós em uma Árvore AVL requerem que os ancestrais do nó inserido/removido tenham seus fatores de balanceamento verificados.
- Para cada operação de inserção ou remoção, verifica-se cada nó ancestral, até a raiz.
- Caso o nó verificado tenha fator de balanceamento < -1 ou > 1 , quatro casos (que na verdade são dois casos com simetria) devem ser considerados para efetuar as rotações e garantir o balanceamento.

Correção de balanceamento em uma Árvore AVL

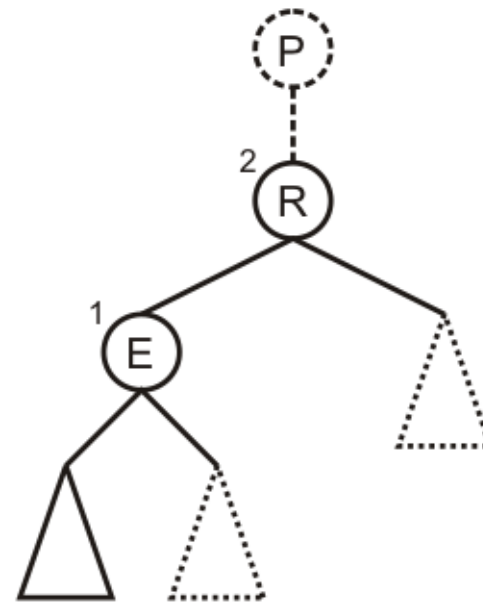
- Seja **R** o nó desbalanceado, **E** seu filho esquerdo, **D** seu filho direito e **P** seu pai (que pode não existir se **R** for a raiz da árvore)
 - Caso 1: Se o fator de **R** é -2 e o fator de **D** é ≤ 0 (right-right),
 - Então: Promover left-rotate em **R**.
 - Caso 2: Se o fator de **R** é -2 e o fator de **D** é $= 1$ (right-left),
 - Então: Promover right-rotate em **D** e left-rotate em **R**.
 - Caso 3: Se o fator de **R** é 2 e o fator de **E** é ≥ 0 (left-left),
 - Então: Promover right-rotate em **R**.
 - Caso 4: Se o fator de **R** é 2 e o fator de **E** é $= -1$ (left-right):
 - Então: Promover left-rotate em **E** e right-rotate em **R**.

Correção de balanceamento

caso 1

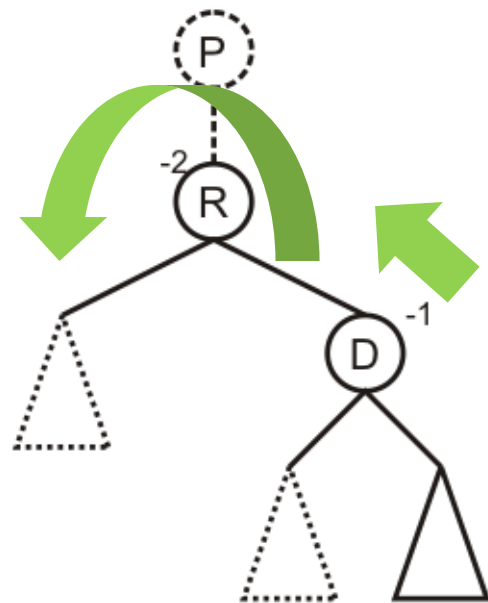


caso 3

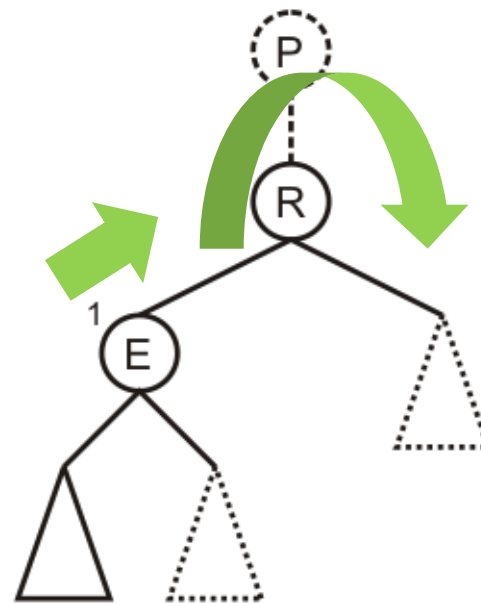


Correção de balanceamento

caso 1

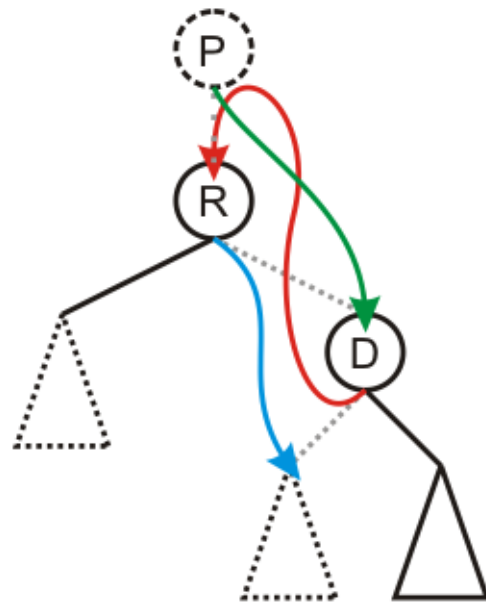


caso 3



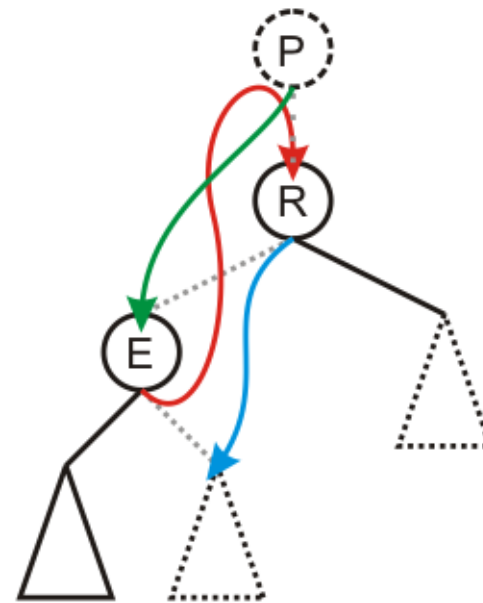
Correção de balanceamento

caso 1



left-rotate em **R**

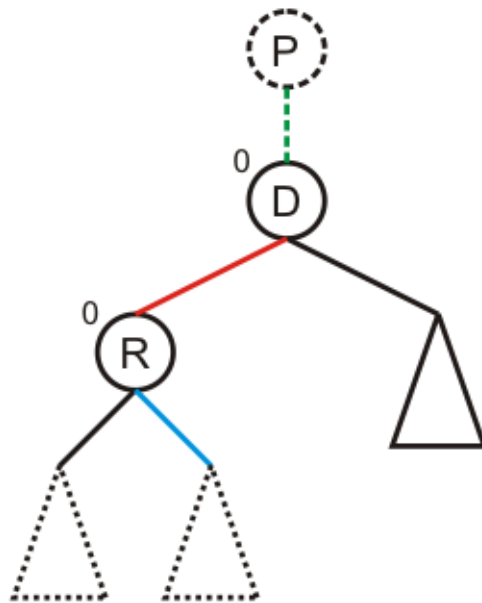
caso 3



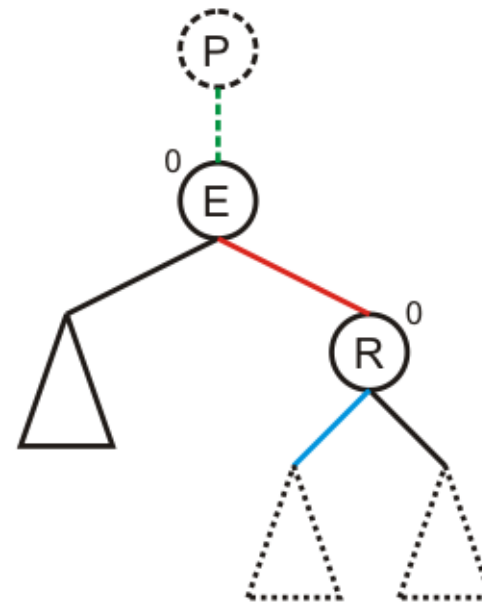
right-rotate em **R**

Correção de balanceamento

caso 1

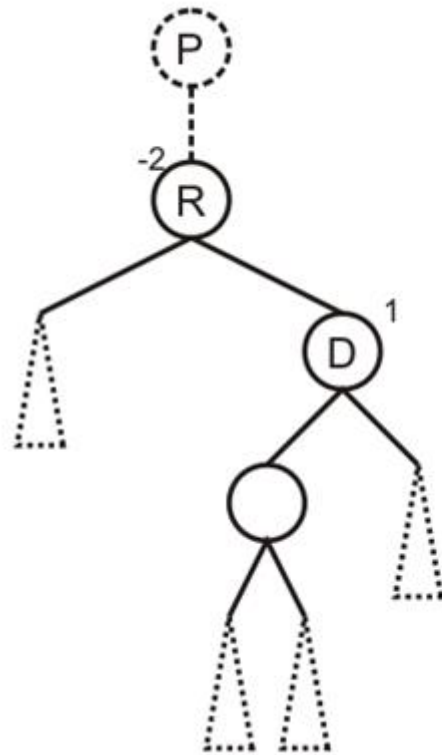


caso 3

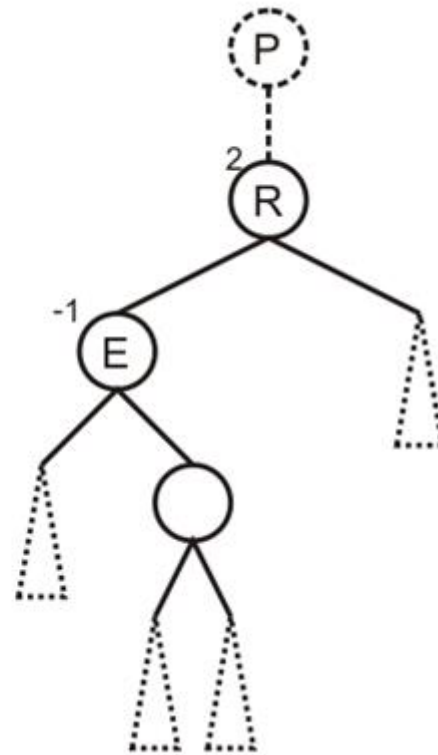


Correção de balanceamento

caso 2

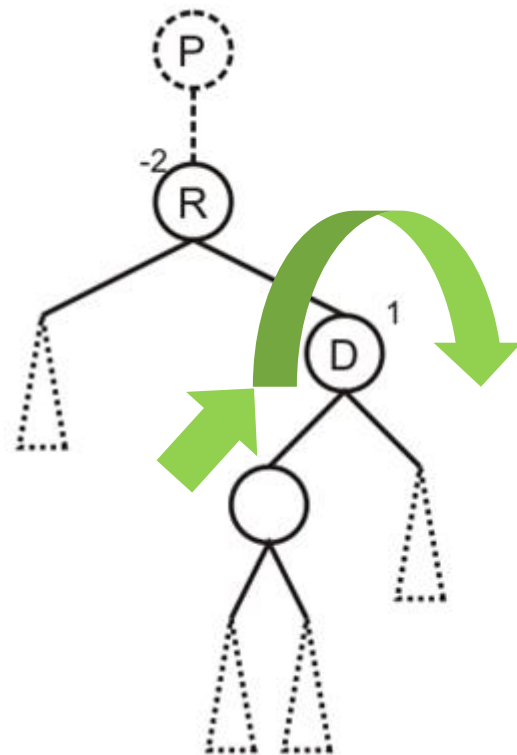


caso 4

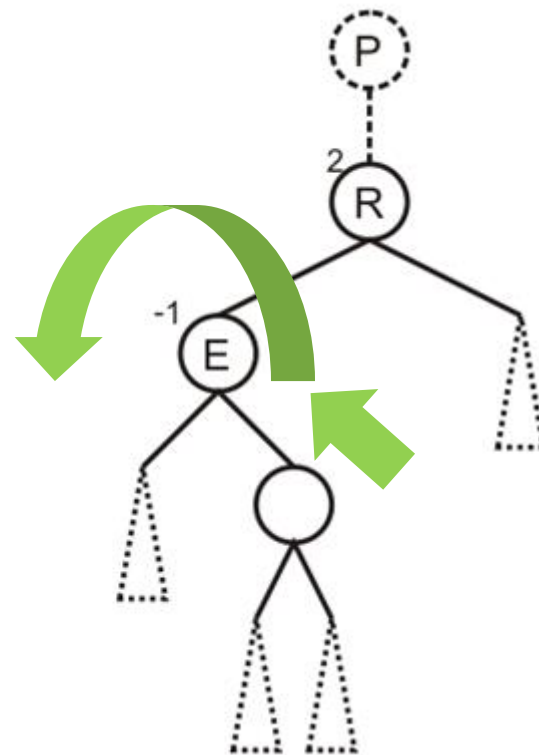


Correção de balanceamento

caso 2

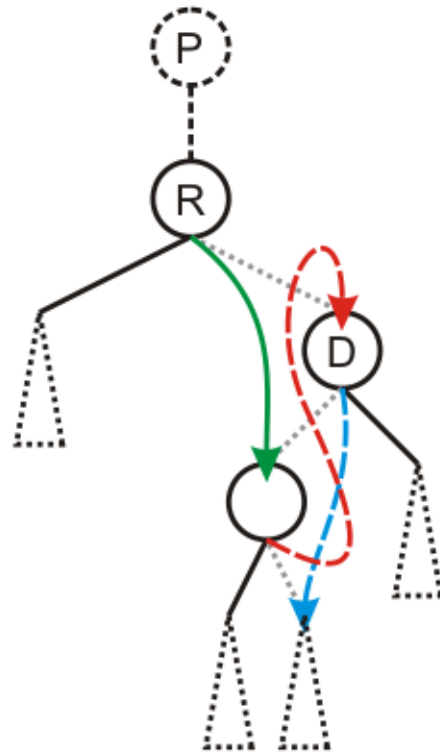


caso 4



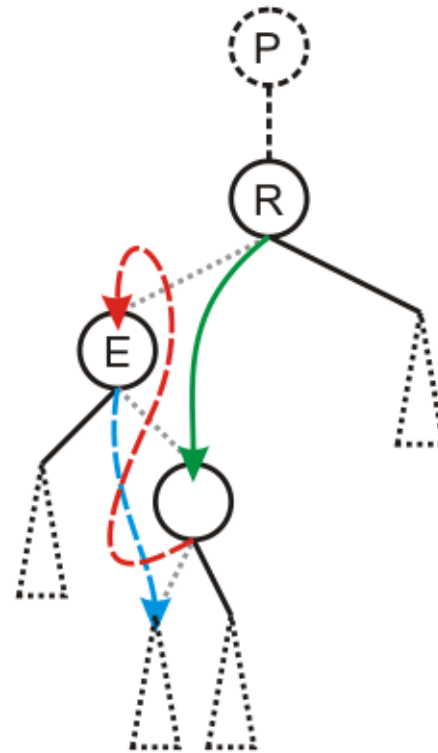
Correção de balanceamento

caso 2



right-rotate em **D**

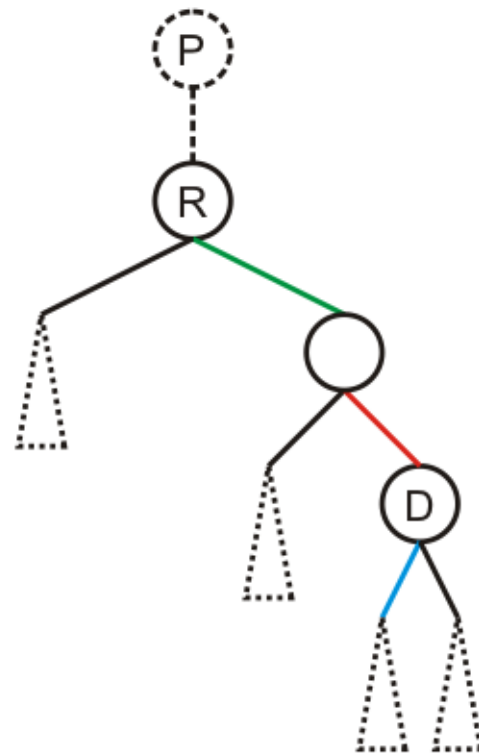
caso 4



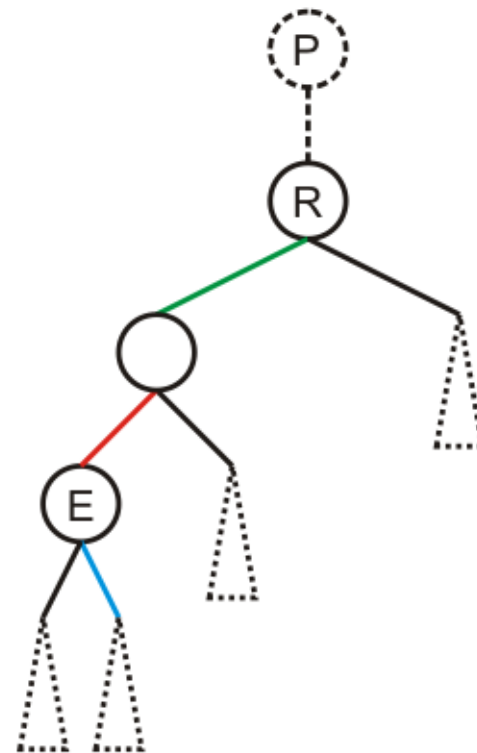
left-rotate em **E**

Correção de balanceamento

caso 2

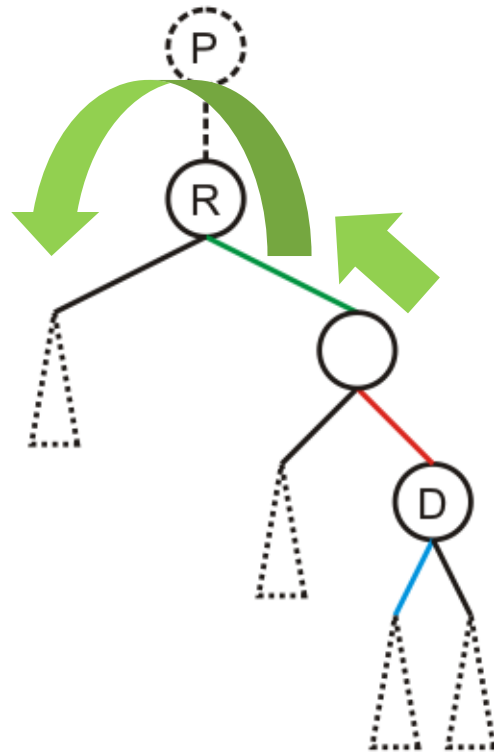


caso 4

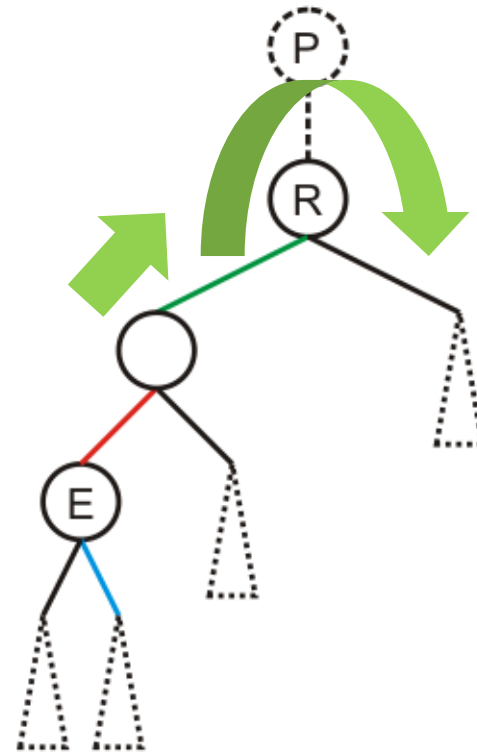


Correção de balanceamento

caso 2

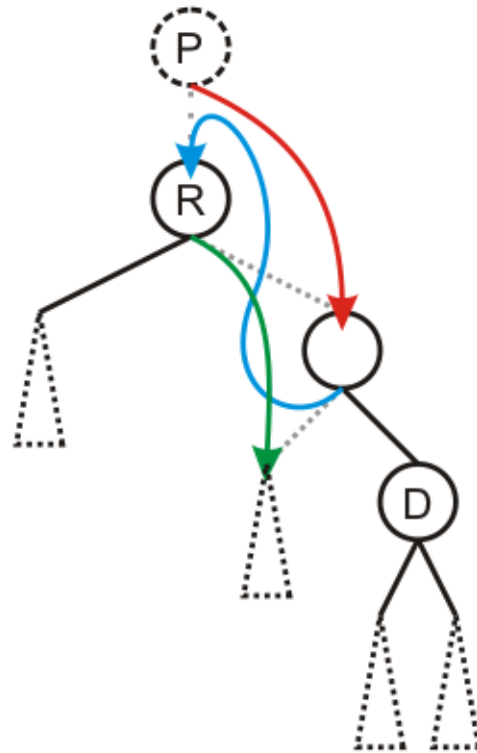


caso 4



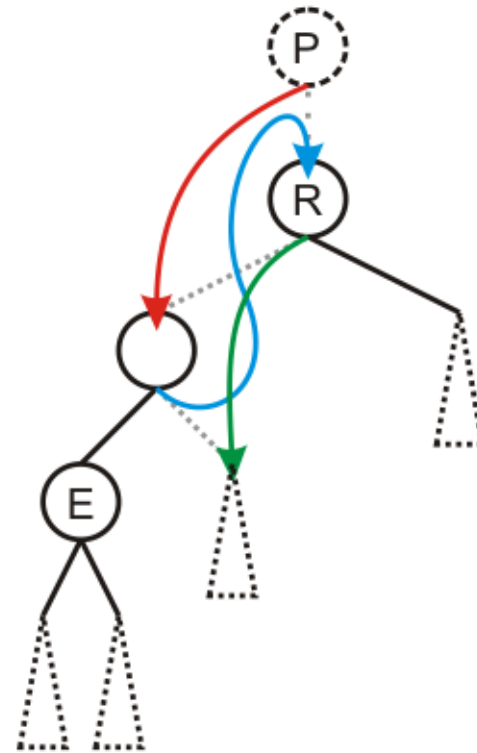
Correção de balanceamento

caso 2



left-rotate em **R**

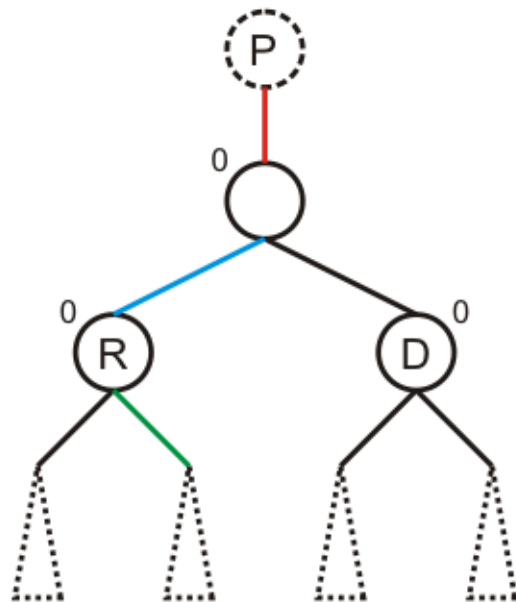
caso 4



right-rotate em **R**

Correção de balanceamento

caso 2



caso 4

